

THE COMPLETE PROJECT DOSSIER



Tlamatini

the one who knows

A self-hosted AI developer assistant with a visual agent-workflow designer, a universal MCP client, and the reach to drive real boards, engines and networks.

89

AGENT TYPES

109

MULTI-TURN TOOLS

29

SKILLS

v1.77.0

CURRENT RELEASE



Created by Angela López Mendoza · @angelahack1

October 10, 2026 · github.com/XAIHT/Tlamatini · MIT license

◆ BEFORE YOU BEGIN

About this dossier

This is the complete dossier of Tlamatini: what she is, how she works, how to use her, every agent she can run, the promises she keeps, and a full inventory of her source tree.

It is written for three readers at once. A **newcomer** can read the first three chapters and know what Tlamatini is and how to start. An **operator** will find the surfaces, the bestiary and the configuration chapters. A **maintainer** will find the architecture, the contracts behind the honesty rules and the complete repository inventory.

Nothing here is typed from memory. Counts, versions, commits, the language inventory and the file tree were derived from Git and the source files when this edition was generated, and the generator then reopened this document to prove that no text overflows or is clipped and that the file tree matches the repository exactly. Configuration values, credentials and private addresses are never reproduced.

This edition	
Generated	October 10, 2026 at 13:33 (UTC-06:00)
Inspected commit	0e6fc72d — Tlamatini's self-repair Image-Interpreter, by Tlamatini's-AutoBot.
Version	1.77.0
Latest published release	v1.77.0
Companion deck	Tlamatini_eXtended_Artificial_Intelligence_Humanly_Tempered.pptx

◆ A note on the numbers

Figures that move with every release — agents, tools, skills, migrations, lines — describe this edition only. Regenerate the dossier and they update themselves.

◆ CONTENTS

Contents

About this dossier	2
I · Who She Is	7
Tlamatini, the one who knows	8
Local control, cloud reasoning	9
Seven subsystems, one assistant	10
The jewels	11
The numbers, derived from source	12
II · How She Works	13
Five layers, from toggle to tool	14
The life of one message	15
The operator loop	16
Never hang, never discard, never lie	17
Every request fits the model it is sent to	18
Every model tuned from formal sources	19
A verdict you can trust	20
Ask Execs: a human gate before risky steps	21
Grounded in your project, and only its text	22
One compiler for the canvas and the chat	23
Built to keep running	24
III · Where You Work With Her	25
Conversation with modes	26
Drop a message from the conversation	27
What remains after Drop	28
Speak beside Send	29
Config ends with Mic	30
One main console, no extra window	31
Measured source behavior	32
The menu bar, reorganized in v1.70.0	33
Read usage while she works	34
Different scopes, clearly identified	35
Report what the account supplies	36
Design agent workflows visually	37
Draw a conversation, then press Play	38
Playback that stays in its own lane	39

Writing and formatting a commentary	40
Comments survive save, open and draft recovery	40
The right editor, from chat or Windows	41
Register, repair and remove both formats	42
Remove the application and its owned workers	43
Repair the boundary, then repeat the real operation	44
More room for the canvas or the log	45
Ready-made prompts, grouped by purpose	45
Read a PDF, then hand her the whole document	46
Every model choice in one place	46
Everything else, without editing files	47
 IV · The Bestiary	48
89 agents, twelve families	49
Flow control	50
Routing and logic	50
Orchestration and intelligence	51
Code, files and shell	52
Documents	53
Web, data and APIs	53
DevOps and infrastructure	54
Quality and security	54
Firmware and engines	55
Desktop and capture	55
Voice, audio and vision	56
Messaging and monitoring	57
 V · Deep Capabilities	58
Other coding agents as her children	59
A universal MCP client	60
Procedures, handed over complete	60
Compose, present, typeset	61
A repair ladder that protects your words	62
Real boards, fail-safe by design	63
Unreal Engine and Blender, live	63
Assessment for authorized targets	64
Windows defence, operated by a human	64
She listens until you stop, and answers in her voice	65
Eyes that report their limits	66
Precise hands on a real desktop	67
Three search tiers, one time budget	67
Read the requested page, report what happened	68

Speed measured with its error bar	68
Reaching people deliberately	69
VI · Promises She Keeps	70
Truth over comfort	71
Fail open or fail safe, on purpose	71
Your agents, your jurisdiction	72
Your data, handled carefully	72
Nothing runs where nobody can see it	73
VII · Install, Configure, Run	74
Five steps to a working Tlamatini	75
Every referenced Python library is accounted for	76
Two modes and their service ports	77
The settings that matter most	78
From source tree to installer	79
One inventory ships every required helper	80
Packaging evidence and its limits	81
Updating herself without losing you	81
One number, from one source	82
VIII · Version and Release Status	83
Usage is verified; rebuild acceptance is next	84
She speaks to you, by name	85
The Model Brain tunes every model	86
Documents and files that tell the truth	87
Smaller fixes that keep the chat moving	88
Graphical notes and resizable Run output	89
Four stop sequences and a clearer Prompt Flow Panel	90
Compact mode and Self-modify become switches	91
Compact mode and readable tables	92
Model lists from the server you configured	93
One card removed from future context	94
Web pages that cannot vanish in silence	95
A context gauge that tells the truth	96
Direct dictation and dependency coverage	97
v1.70.0: the Prompt Flow Panel	98
How the line arrived here	98
Tag, commit and publication, stated plainly	100
IX · Repository Facts	101
The repository in numbers	102

Effective lines by language	103
Every language, counted	104
Authored and vendored	105
The largest source files	105
Recent commits	107
How these numbers were produced	108
 Appendix A · The complete tracked file tree	 109

CHAPTER

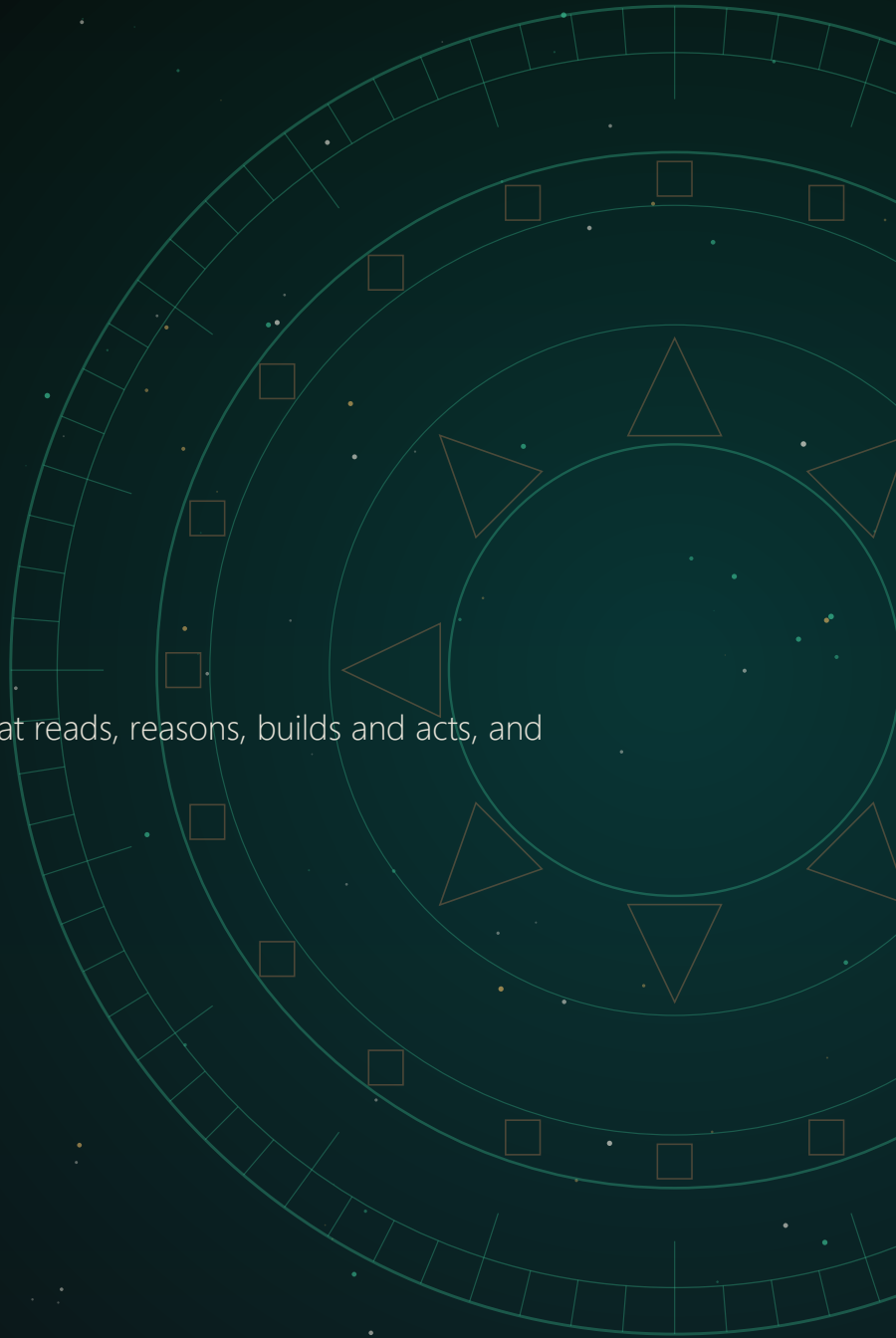
I

Who She Is

A self-hosted developer assistant that reads, reasons, builds and acts, and lets you design how.

IN THIS CHAPTER

- ◆ Tlamatini, the one who knows
- ◆ Local control, cloud reasoning
- ◆ Seven subsystems, one assistant
- ◆ The jewels
- ◆ The numbers, derived from source



◆ IDENTITY

Tlamatini, the one who knows

Tlamatini is a self-hosted AI developer assistant for Windows 10 and 11: a Django application that converses, retrieves, plans, runs agents, and lets you draw whole agent workflows on a canvas.

Her name is Nahuatl. In the Nahua world a *tlamatini* was a sage, literally “one who knows”: a keeper of the painted books who carried knowledge and taught others to see. The name is a promise about behaviour. She should know what she is doing, say what she actually did, and never pretend.

Tlamatini was conceived, designed and built by **Angela López Mendoza** (@angelahack1), who gave her the name. Her birthday is October 29, 2025. She is female by design, including her spoken voice, which is always a female voice, and she refers to herself as *she*.

She is free, open-source software under the MIT license, published at github.com/XAIHT/Tlamatini. The repository carries the application and every one of its 89 workflow-agent programs as plain, readable Python.

Creator

Angela López Mendoza · @angelahack1

Born

October 29, 2025

Platform

Windows 10 and 11 · Python 3.12.10 · Django 5.2.15

License

MIT: free and open source

◆ WHERE THINGS RUN

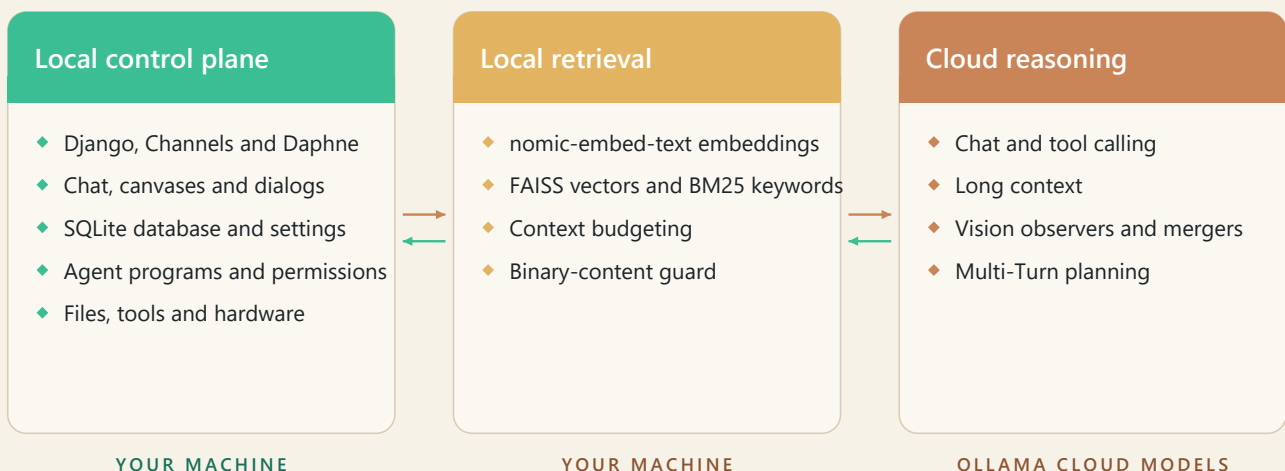
Local control, cloud reasoning

“Local-first” describes where control lives: the application, database, workflows, credentials, agent code and the small embedding model stay on your machine, while the demanding reasoning is done by cloud models reached through Ollama.

Three planes cooperate. The **local control plane** is the Django application with its chat, canvases, SQLite database, agent programs, permissions, files and hardware connections. **Local retrieval** builds embeddings with `nomic-embed-text` so project context is found quickly. **Cloud reasoning** — configured `:cloud` models served through Ollama — performs chat, tool calling, long context, vision and Multi-Turn planning. A smaller local model also works: a request it cannot hold is sent in Compact mode, and the Model Brain tunes every model from its maker's published settings.

Because the complete experience was engineered around Ollama's cloud-model capacity, an active **Ollama Pro plan or higher** is part of the intended system requirements. This is an independent technical recommendation: Tlmatini and XAIHT are not sponsored by, affiliated with or compensated by Ollama.

Whatever you send to a configured cloud model is processed by that provider. Direct Claude API use, external coding-agent CLIs and remote MCP servers are configured separately, so you decide what material leaves your machine.



◆ THE PILLARS

Seven subsystems, one assistant

Everything Tlamatini does rests on seven cooperating subsystems.

Hybrid RAG

FAISS vectors, BM25 keywords, metadata extraction, context budgeting and a binary-content guard keep answers grounded in your real project.

Multi-Turn

A planner and a tool loop of up to 4,096 iterations over the full enabled surface of 109 built-in tools, self-healing steps and Compact mode for small models.

Agentic Control Panel

89 drag-and-drop agent types wired into runnable, savable `.flw` workflows.

Prompt Flow Panel

New in v1.70.0: prompts, decisions and embeddings drawn as a diagram, saved as `.fpmt`, and played against your models.

ACPX

14 external coding-agent CLIs, from Claude Code to Codex and Gemini, spawned as children and brokered through 12 tools.

External MCPs

A universal client for any MCP server over four transports, with up to five servers active at once.

Skills

29 `SKILL.md` runbooks, from code review to MCP onboarding, handed to the model as complete procedures.

◆ WHAT SETS HER APART

The jewels

Coding assistants edit text files. Tlamatini does that too, then reaches into engines, boards and networks, and lets you wire all of it together visually.

Visual workflow design

89 agent types on a canvas you connect, validate, run, pause and save.

Unreal Engine

Drive a live UE5 editor from chat or canvas: actors, Blueprints, levels, assets and materials.

Blender

Scenes, objects, materials, renders and raw bpy code over the official add-on socket.

Real firmware

Scaffold, build and flash STM32, ESP32, Arduino and ESPHome boards behind fail-safe preflights.

Whole projects

Find, read, edit and rebuild entire codebases with grounded retrieval.

Any MCP server

One client for the MCP ecosystem: stdio, streamable HTTP, SSE and WebSocket.

Security assessment

Authorized Kali, ProjectDiscovery and nmap work, driven from chat.

Blue-hat defence

An administrator-run Windows monitoring and response toolkit.

◆ AT A GLANCE

The numbers, derived from source

Every figure below was counted from the repository while this dossier was generated.

The built-in Multi-Turn surface is 20 core tools + 67 wrapped agent launchers + 12 ACPX and Skill tools + 10 External-MCP supervisors = **109 tools**. Healthy, active external MCP servers add their own `ext__<server>__<tool>` tools on top.

The application ships 56 JavaScript modules, 20 stylesheets and 6 page templates, 213 database migrations and 87 pinned Python requirements.



CHAPTER

II

How She Works

From a keystroke in the browser to a verified result on disk: the layers, the loop and the verdict.

IN THIS CHAPTER

- ◆ Five layers, from toggle to tool
- ◆ The life of one message
- ◆ The operator loop
- ◆ Never hang, never discard, never lie
- ◆ Every request fits the model it is sent to
- ◆ Every model tuned from formal sources
- ◆ A verdict you can trust
- ◆ Ask Execs: a human gate before risky steps
- ◆ Grounded in your project, and only its text
- ◆ One compiler for the canvas and the chat
- ◆ Built to keep running



◆ ARCHITECTURE

Five layers, from toggle to tool

A request passes through five layers, each with one responsibility, between the browser above and the model back-ends below.

Above the layers sit the browser pages — the chat, the Agentic Control Panel and the Prompt Flow Panel — talking to Django Channels over WebSockets under the Daphne ASGI server. Below them sit the language-model back-ends: Ollama (local and cloud), Anthropic Claude and Qwen vision.

A row in the database only switches a capability on or off. It never creates one: an MCP provider needs its runtime service and context chain, and a tool needs its implementation.

BROWSER

Chat page · Agentic Control Panel · Prompt Flow Panel · WebSockets

1

Persisted toggles

Mcp, Tool, Agent and Skill rows decide what is enabled; the Configure dialogs flip them.

2

Runtime MCP services

System-Metrics (WebSocket :8765) and Files-Search (gRPC :50051) start with the application.

3

Context sidecars

Chains that decide whether system or file context is needed and inject it into the payload.

4

Answer chains

Basic, history-aware RAG and unified-agent chains compose the prompt and produce the answer.

5

Unified-agent tools

20 core, 67 wrapped agents, 12 ACPX/Skill and 10 External-MCP supervisor tools.

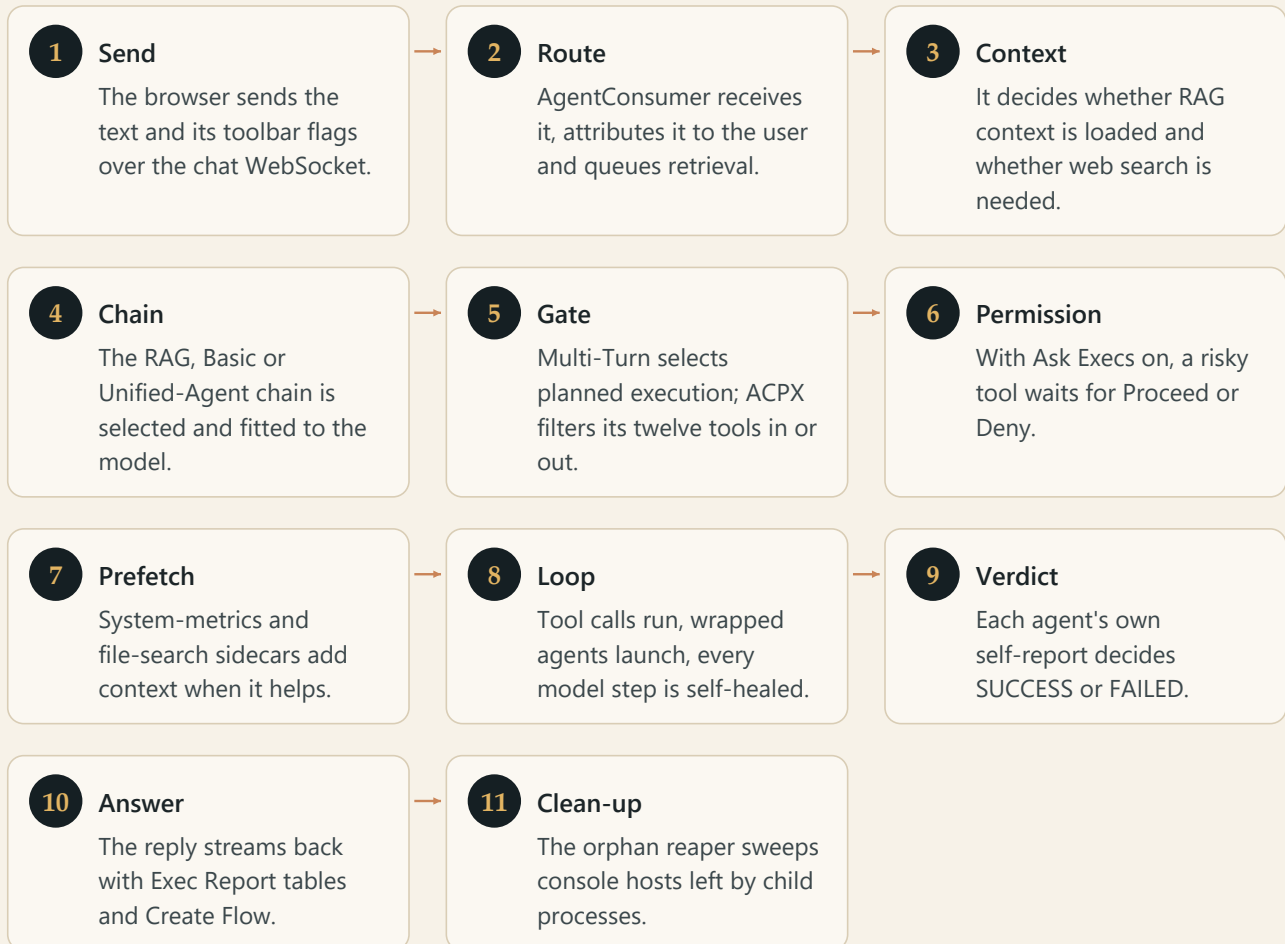
MODELS

Ollama (local and cloud) · Anthropic Claude · Qwen vision

◆ REQUEST JOURNEY

The life of one message

Eleven steps take a message from the browser to a verified, reported result.



◆ MULTI-TURN

The operator loop

With Multi-Turn checked, Tlamatini stops being an advisor and becomes an operator: she plans, calls tools, reads their results and continues until the task is done.

A global execution planner builds a request-scoped plan and capability hints, but the executor binds the **full enabled tool surface**: every enabled tool, wrapped agent and skill, with ACPX still filtered by its own checkbox. An earlier design bound only a narrow planner subset and could starve the loop of the one tool it needed.

The loop runs up to 4,096 iterations. Identical wrapped-agent calls in one request are deduplicated, a repetition breaker nudges a model that circles, and each wrapped agent runs in its own isolated runtime folder, so templates are never modified.

When at least one agent succeeded, the answer carries a **Create Flow** button. It turns exactly the successful tool calls into a canvas-loadable `.flw` workflow, normalized and secret-redacted by the backend, so a conversation can become a reusable workflow.

Step-by-Step

One concrete action at a time; she waits for your READY or output.

Ask Execs

A human Proceed/Deny gate before each risky step.

Exec Report

One table per agent family, one row per real tool call.

ACPX

Adds the external coding-agent and Skill tools to the surface.

◆ SELF-HEALING

Never hang, never discard, never lie

Every model call inside the loop goes through a per-request self-healing invoker that switches to a genuinely different tactic each time a transient failure strikes.

Each attempt runs under a watchdog. The shipped configuration allows 900 seconds, deliberately longer than the 600-second model-client timeout, so the watchdog never abandons a call the client would still finish; the Cancel button is polled every quarter of a second, so she never blocks forever. The ladder cycles through six tactics and keeps going up to 4,096 of them. Only you, with Cancel, stop her.

If the ladder is ever exhausted after agents already ran, she finishes gracefully from that real work: the Create Flow button and the Exec Report survive, and the answer says what happened.

Recovery is narrated live. Status lines such as “Tactic #3 ... I will NOT hang” stream into the chat as they happen, and a SELF-HEALING NOTE opens the final answer.

1	normal	The full request again.
2	retry	The same tool-bound request; most blips clear here.
3	patient retry	Wait, then retry with extra patience.
4	trim context	Keep the system messages and the last twelve.
5	minimal	Keep the last six messages; tools stay bound.
6	plain summary	Last resort: summarize truthfully what was gathered.

◆ Real bugs are not hidden

Only transient failures — timeouts, resets, HTTP 429 and 5xx — trigger new tactics. A deterministic error such as a bad schema is raised at once so it gets fixed.

◆ COMPACT MODE

Every request fits the model it is sent to

A model whose real window cannot hold the complete request receives a compact one; a model that can hold it receives exactly the request it always did.

Ollama does not refuse an oversized request for a local model: it keeps the end and answers from that. A local qwen2.5 whose 32,768 tokens were split between two request slots read 16,386 tokens of a request of about 52,000, lost the live system metrics, and answered a CPU question with the time.

Ollama's own count is now the proof. When the characters sent cannot fit the tokens Ollama says it read, the request was cut, and that count is learned as the model's real window. A local model's reported context length is never taken as its per-request window, because parallel request slots divide it.

A request that fits the usable window is sent byte for byte, as before. Otherwise Compact mode keeps System-Metrics and Files-Search, whose context rides inside the question, and binds only the rows you tick. Since v1.75.0 that is a toolbar switch: ticking it unticks every Configure row but those three and pauses External MCPs, and each row shows its price. The system prompt is rebuilt by rule priority and closed by a seven-rule note, and history keeps the six newest messages. A first step that Ollama still cut is re-fitted and resent before any tool runs.

Full

A model that can hold the request receives it unchanged, byte for byte.

Compact

A toolbar switch: only the rows you tick are bound, each with its price.

Locked

A model too small for everything turns Compact on by itself and locks it.

Your choice

`context_compact_mode` is auto by default, or always or never, obeyed exactly.

◆ **It fails open to the full request**

Every part of the fitter fails open: when anything is uncertain, the complete request is sent exactly as before.

◆ MODEL BRAIN

Every model tuned from formal sources

Since v1.76.0 each model receives its own sampling, thinking level and context size, taken from formal sources, instead of one fixed set of values for every model.

The values come, in this order, from your own `model_brain_overrides`; a built-in knowledge base whose every profile cites its vendor model card or Hugging Face `generation_config.json` (GLM-5, DeepSeek-V4, MiniMax-M3, Gemma 4, Qwen 3.5, Qwen 2.5, gpt-oss, Kimi, Mistral Large 3 and Nemotron 3); a profile learned once, in the background, for a model she does not know yet and kept outside the installation so it survives updates; the parameters Ollama publishes for the model; and finally the model's own defaults. The requested context never exceeds what the model can really read.

Inside one answer, between tool calls, a thinking model gets its own earlier reasoning back, as its maker requires, instead of starting over at every step; it is dropped between your questions, and the context gauge counts it. In Config ▶ Models, Save opens an Auto-tuning dialog that tunes every configured model in front of you, with the source of each value.

The stall clocks that abandoned a slow model and started again from scratch were removed. Measured with Ollama's own numbers, reading even a very long prompt is not the bottleneck on Ollama cloud: the time is the reasoning the model generates, and a clock cannot tell a dead call from a model that is still thinking. The self-healing watchdog and the client time limit remain.

Formal sources

Your overrides, cited profiles, learned profiles, Ollama, model defaults.

Real context

The requested window never exceeds what the model can read.

Reasoning kept

Thinking models keep their train of thought between tool calls.

Auto-tuning

Config ▶ Models ▶ Save shows each model tuned, with its sources.

No stall clocks

A slow model is never thrown away and restarted.

Your choice

`model_brain: off` restores the legacy fixed sampler exactly.

◆ No model name in the code

A new model gets its values from the knowledge base, research, Ollama or its own defaults, never from a branch written for one model name, and a value you set always wins.

◆ EXEC REPORT

A verdict you can trust

An exit code is one bit. An agent's own structured self-report is a typed record, and it outranks the exit code.

`agent_verdict.py` parses each agent's `INI_SECTION` block into a typed tree and runs an ordered rule table over it. There is no model call and no heuristic, so the same input always yields the same colour. The status vocabulary is closed: five disjoint sets, united as `KNOWN_STATUSES`, and a repository-wide test rejects any status token an agent invents.

A read-only diagnostic that reports a problem has succeeded, because the finding is the deliverable. Degraded work, work not done and agent errors stay red. A red row therefore means that no clean requested deliverable exists, never merely that a check found something.

Rule	Fires when	Verdict
R1	The agent wrote no self-report	Exit code decides
R2	The agent declares error or failed	FAILED
R3	Work not done: refused, not_found, engine_unavailable ...	FAILED
R3b	Degraded: tokens_only, compiled_with_errors, operator_required ...	FAILED
R4	A diagnostic completed: invalid, findings, no_matches, listed ...	SUCCESS
R5	An explicit success or ok flag	That flag
R6	A non-zero errors count	FAILED
R7	Nothing decisive and a non-zero exit code	FAILED
R7b	A named completion: ok, created, sent ...	SUCCESS
R8b	A status token outside the vocabulary	SUCCESS, flagged
R8	No failure signal at all	SUCCESS

◆ Why R4 outranks R5 and R6

A linter that worked perfectly reports `status: invalid`, `success: False` and `errors: 2` together. The last two describe the document, not the agent.

◆ PERMISSION

Ask Execs: a human gate before risky steps

With Ask Execs on, the executor blocks before each risky tool and waits for Proceed or Deny in the browser. A single Deny halts the whole chain and leaves a red "Execution interrupted" banner.

The gate is an explicit allowlist, chosen tier by tier. Every doubtful path resolves to Deny: a failed prompt, a Cancel or a disconnected browser never lets a risky tool run unconfirmed. Unchecking Ask Execs during a run relaxes that run; checking it again re-arms the prompts.

Tier	Prompted	Agents
Command and script runners	Yes	Executer, Pythonxer, SSHer, Kalier, Dockerer, Kubernetes, SQLer, Mongoxer, Gitter, Jenkinser, PSer, J-Decompiler
A · destroys or overwrites	Yes	Deleter, Mover, File-Creator, Editor, De-Compressor, unzip, PDFer, LaTeXer
D · remote systems	Yes	SCPer, Apirer, Nmapper, Discoverer, Crawler, NetSpeed-Calculator
B · messaging	No	Emailer, Whatsapper, Telegrammer, Zavuerer, Instant Messaging Doctor: sending is the model's own judgement
C · desktop and hardware	No	Keyboarder, Mouser, Windower, Playwrighter, firmware agents, Blenderer, Unreal: visible while they act

◆ RETRIEVAL

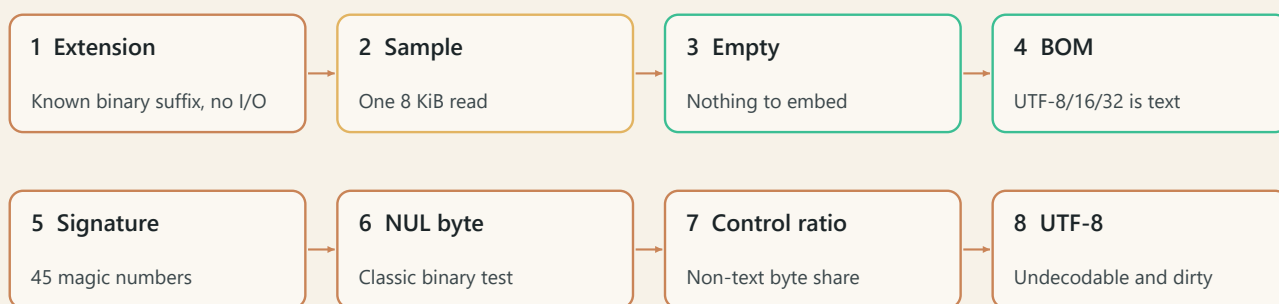
Grounded in your project, and only its text

Hybrid retrieval combines FAISS vectors with BM25 keywords, extracts metadata, budgets the context and falls back gracefully when a chain cannot retrieve.

When you load a directory or file, that loaded context outranks her self-knowledge: a request to “summarize the project” is answered from your project, not from a description of herself.

Before any file is embedded, a binary-content guard screens its bytes in a short-circuiting cascade that reads at most one 8 KiB block. Binary files are dropped exactly like a name you chose to omit, and every drop is named in `tlamatini.log` under `---` `[BINARY-GUARD]`.

The guard fails open: any doubt means “load it as text”, because silently deleting real context is worse than embedding one odd file. The byte-order-mark stage runs before the NUL-byte stage, or every UTF-16 document would vanish.



Copper stages can drop a binary file; jade stages can only keep a file as text. Any doubt keeps the file.

◆ FLOW COMPILER

One compiler for the canvas and the chat

Canvas snapshots and chat-generated drafts compile through the same Agent Contract registry before any file reaches the session pool.

Each agent's contract declares which configuration field a connection on each slot writes, which fields Parametrizer may read, which paths are secrets to redact, and whether it is a singleton, long-running or never starts its targets. Both surfaces normalize into one **FlowSpec** (schema version 2).

The canvas Start button compiles the live snapshot in write mode, so an edited but unsaved flow runs under the same validation as a freshly loaded file. Validate uses a dry run that previews the compiled configuration without touching disk.

Agents that feed Parametrizer emit one atomic **INI_SECTION** block per result. 53 agents currently declare structured outputs.

```
INI_SECTION_APIRER<<<
url: https://example.org/status
status: 200

{"service": "ok"}
>>>END_SECTION_APIRER
```

◆ RELIABILITY

Built to keep running

Small, deliberate mechanisms keep the core alive under the conditions that used to stop it.

Console shield

The log file is written first; console output goes through a bounded queue, so clicking the console cannot freeze her. Since v1.76.0 lines are coloured by level.

Orphan reaper

Three tiers sweep dead descendants and orphaned console hosts after tools, after answers and at shutdown, without ever raising into the chat.

Per-user log lines

Lines carry a five-character tag such as [a3] so concurrent users and turns never blur.

Clean Ctrl+C

The signal handler only sets an event; a worker cleans up, a watchdog guarantees exit, and a second Ctrl+C exits at once.

Configurable port

`django_port` moves the web port with no rebuild and falls back to 8000 on any bad value.

Temp and Templates

Scratch files live under the application's Temp folder; scaffolded projects default to its Templates folder.

CHAPTER

III

Where You Work With Her

The chat, the two canvases, the catalog and the dialogs that operate everything else.

IN THIS CHAPTER

- ◆ Conversation with modes
- ◆ Drop a message from the conversation
- ◆ What remains after Drop
- ◆ Speak beside Send
- ◆ Config ends with Mic
- ◆ One main console, no extra window
- ◆ Measured source behavior
- ◆ The menu bar, reorganized in v1.70.0
- ◆ Read usage while she works
- ◆ Different scopes, clearly identified
- ◆ Report what the account supplies
- ◆ Design agent workflows visually
- ◆ Draw a conversation, then press Play
- ◆ Playback that stays in its own lane
- ◆ Writing and formatting a commentary
- ◆ Comments survive save, open and draft recovery
- ◆ The right editor, from chat or Windows
- ◆ Register, repair and remove both formats
- ◆ Remove the application and its owned workers
- ◆ Repair the boundary, then repeat the real operation
- ◆ More room for the canvas or the log
- ◆ Ready-made prompts, grouped by purpose
- ◆ Read a PDF, then hand her the whole document
- ◆ Every model choice in one place
- ◆ Everything else, without editing files

◆ THE CHAT PAGE

Conversation with modes

The chat page is where you talk to her. Up to eight toolbar switches decide how each request runs.

Beside the conversation sits a canvas that shows code, text and PDFs. Paste a screenshot with Ctrl+V or drop images on the chat: she saves them to her Temp folder and writes the full path into your message, ready for Image-Interpreter. A context ring between the toolbar and the message box shows how much of the model's window the real request uses, measured by the backend in bytes. Tokens use Ollama's own `prompt_eval_count` for the request; an estimate is labeled until a measured count arrives. When Ollama cut a request the ring says so, and an answer built from a cut request carries a CONTEXT-WINDOW warning.

Her avatar speaks answers aloud when you ask. Since v1.72.3, clearer syllable movement and faster closure make her lips easier to follow, with opening capped to her portrait. After login, pressing Enter on the welcome page takes you straight to the chat.

Answer tables stay readable with every model: only a cell whose text would fall below 3:1 contrast is recoloured, while readable tables, Exec Report tables and gradient backgrounds keep their colours.

Switch	What it does
Multi-Turn	Planned, tool-calling operator mode; unchecked means direct one-shot answers.
Exec Report	Appends per-agent execution tables; available while Multi-Turn is on.
ACPX	Adds the twelve ACPX and Skill tools to the bound surface; off by default.
Ask Execs	Proceed/Deny before each risky tool; a Deny stops the chain.
Step-by-Step	One concrete action at a time, waiting for your reply before the next.
Internet	Allows a web search to add context to the answer.
Compact mode	Binds only the rows you tick; locked on when the model cannot hold all.
Self-modify	Sends her self-knowledge; only in self-modify builds, locked off if too big.

◆ CHAT HISTORY

Drop a message from the conversation

Every message card has Drop beside Copy, for your prompts and Tlamatini's answers.

Press Drop and read the themed confirmation. Cancel starts focused, so Enter cancels by default. The red Drop button confirms deletion. Escape and the close button also cancel. The saved message disappears from the chat and the database history used for future requests.

The other messages stay. Dropping only a prompt leaves its answer; dropping only an answer leaves its prompt. Remove both cards to remove the exchange. A user prompt also takes its directly following Referenced Rephrase rows, so a reload cannot show those duplicate words.

Connected tabs for the same user remove the saved cards after the server confirms. A status, error or retry card without a saved message id only disappears from the current screen. The connection refuses Drop while it is answering, and a disconnected chat asks you to reconnect.

Choose

Drop sits beside Copy on both user and assistant message cards.

Confirm

Cancel starts focused. Red Drop confirms; Escape and close cancel.

Remove

Delete the saved message from chat history. Its paired prompt or answer stays.

Continue

The next request reads the updated history without a reconnect.

◆ MEMORY AND EFFECTS

What remains after Drop

Deleting a chat card changes future conversation context. Separate saved notes and completed work remain.

Each request reloads up to eight newest AgentMessage rows through DBChatHistoryLoader, then filters status and rephrase content. No cached conversation summary or LangGraph checkpoint needs clearing. Removing a row lets the window reach an older row when one exists.

An enabled External MCP memory tool stores notes separately from AgentMessage. Drop does not erase that knowledge graph; ask Tlmatini to forget the saved note too. Retained messages may also repeat a fact from a deleted card, so Drop is not a guarantee of complete forgetting.

Files created, messages sent, agent runs and Exec Report rows remain real completed work. Dropping their prompt or answer cannot undo those effects. A Create Flow button belongs to its answer card and disappears with it, so download a needed .flw before deleting the card.

History

Every request reloads a window of up to eight newest database rows.

Saved notes

External MCP memory is separate. Ask her to forget its note too.

Other messages

The paired answer or prompt stays and may repeat the same information.

Completed work

Files and sent messages remain. Save a needed flow before dropping its card.

◆ DIRECT VOICE INPUT

Speak beside Send

The microphone captures first; no model tool-selection step stands between the click and recording.

Click Mic beside Send, wait for Listening, and speak into the microphone on the Tlamatini host. Real input level, elapsed time and the silence countdown stay in the chat. After the configured silence window (shipped default 3.5 seconds), recognition runs and the text is appended to the existing draft. Config > Mic selects automatic normal-form submission or an editable draft for manual Send.

Review mode starts no task until Send. A second click or Escape cancels; empty or failed recognition sends nothing. Current modes and permission behavior remain in force. The avatar acknowledges dispatch through browser speech and respects Silent mode.

Click

Direct Whisperer start; the host microphone opens only for a recording.

Listen

Live samples drive the in-chat indicator and the configured silence gate.

Recognize

Local faster-whisper or the explicitly configured cloud speech engine.

Send or review

Submit once with current modes, or review the editable draft and press Send yourself.

◆ CONFIGURATION

Config ends with Mic

Choose automatic Send or an editable draft, using the same dialog style as the rest of Tlamatini.

The button label is Mic. Config > Mic follows Voice and controls what happens after recognition. Send automatically is the default. Keep in the chat input restores editing without launching a task or processing acknowledgment; correct the words, then press Send.

Preferences are saved in this browser for the next recording. Each recording owns a snapshot. Unset fields inherit the current Whisperer template; Reset then Save restores that inheritance. Cancel, close and Escape discard unsaved edits; outside clicks leave the dialog open. Config > Voice controls avatar playback; Config > Models > Speech selects the recognizer.

Input

Configured/default/listed host device, refresh and software gain from 0 to 300%.

Sound gate

Silence 0.3-20 seconds, cap 5-600 seconds, adaptive or manual sensitivity.

Recognition

Language, English translation, capture rate/channels, local beam size and VAD.

Guardrails

Validate settings before capture; removed or ambiguous saved devices cannot silently change inputs.

◆ INTERNAL SERVICE

One main console, no extra window

Developer visibility requirements do not become additional product windows.

The authenticated same-origin `/ws/chat-voice/` connection prepares one resident worker per web process. Source/carried Python runs `chat_worker.py`; the frozen Django process stays free of the speech ML stack. A private token-authenticated loopback socket carries control and result frames. Preparation does not open the microphone.

No PowerShell, conhost, focus activation or console-visibility prerequisite is involved. The worker's `stdout/stderr` flows through `agent.chat_voice_runtime` into the main console/log. Failed startup and shutdown reap the child. Config reloads per job; compatible models are cached, and late cancelled results cannot submit.

UI ownership

The composer displays recording and transcription; no second console.

Configuration

Model/template inheritance plus validated, per-recording Mic capture overrides.

Boundary

The direct button is input, not a wrapped tool or an Exec Report row.

Release

Ship consumer/runtime/settings modules, carried validator and worker, Mic JS/CSS and template.

◆ VERIFICATION SCOPE

Measured source behavior

Dated source checks are evidence, not a claim that an older installer contains the change.

On September 27, 2026, 108 voice regressions and 42 existing dialog-theme tests passed. The worker refreshed 16 host inputs and restarted twice without extra windows or focus changes. Earlier real host capture yielded samples in 297 ms after direct start; cancellation and normal worker exit were observed. Two startup/shutdown cycles created no additional visible windows and left foreground focus unchanged.

The updated headed-browser suite passed 62 controlled-transport checks, including both modes, settings persistence, shared control styles and narrow layouts. Earlier cached recognition of a 5.768-second synthetic speech file took 0.203 seconds, versus 11.094 seconds on first decode. These are separate observations, not end-to-end latency guarantees. No new live Ollama task or rebuilt installer was certified by those checks.

Current source

108 voice regressions, 42 theme tests and real worker metadata/lifecycle probes.

Browser evidence

62 headed checks with controlled transport; separate from live ASR.

Limits

Cold model loading, provider latency and hardware vary; no zero-latency promise.

Detailed guide

docs/chat-microphone-design.md records configuration, protocol and evidence.

◆ NAVIGATION

The menu bar, reorganized in v1.70.0

A new Panels menu opens the two canvases, and every configuration dialog lives under Config.

Menu	What you find there
Open · Save	Load a file into the canvas or save its contents.
Context	Load a directory or file as context, and set file-type omissions.
Panels	Agentic Control Panel, Prompt Flow Panel and, for staff, the Admin Panel. It stays usable while she is busy.
ACPX-Skills	Browse, configure, diagnose and reload the skill catalog.
External	The External MCPs dialog: catalog, activation and runtime readiness.
Config	Configure MCPs, Configure Agents, Models, URLs, Contacts, Access Keys Wizard, Voice, then Mic (dictation behavior and capture preferences).
DB	WAL-safe database backup and staged replacement.
Reconnect · About	Rebuild the chat connection; version, author credits and Check for updates. Current source also adds Usage, available while a request runs.

◆ ABOUT → USAGE · CURRENT SOURCE

Read usage while she works

ABOUT USAGE follows the shared dialog theme and remains available during a long answer.

Open About → Usage, then choose Cloud credits, Chat activity or Models. The first tab reads the configured Ollama account; the other views identify measured main-chat activity for the signed-in Tlamatini user. Recording begins with this feature and does not reconstruct earlier history or include child agents and other applications.

The desktop dialog is 80vw by 74dvh, capped at 1440 pixels wide: at most 59.2% of the client area. Narrow screens use 94vw by 63dvh. Text follows normal dialog sizing, buttons have clear borders and the body scrolls between fixed controls. Fractional displays have at most two decimal places. Colored bars offer keyboard-focus and pointer tooltips plus exact tables.

Cloud credits

Reported balance, monthly usage, allowance and refill weeks/date.

Chat activity

Your measured calls, input/output tokens and daily trends.

Models

Recorded model activity and peak UTC dates; server inventory alongside it.

While busy

Open and refresh without interrupting the model request.

Normal size

Under 60% client area; ordinary typography and clear buttons.

Accessible

Keyboard tabs, chart tooltips, focus trap and normal Escape/Close behavior.

◆ ACCOUNT DATA

Different scopes, clearly identified

Monthly credits, rolling cloud usage and recorded local activity answer different questions.

The authoritative `/api/balance` response supplies included allowance and remaining balance, purchased credits and the UTC refill period. Used credits equal allowance minus included balance; total available adds purchased credits only when both values are reported. Plan names never supply an assumed allowance. Refill weeks/date come from the returned period, not an assumed 30-day cycle.

Cloud `/api/usage` supplies rolling 7/30-day history, including today's partial UTC day. That spend is not monthly credits used. Cached input is already part of input. The local ledger's 7/30 calendar dates include today and have a narrower measurement scope.

Ollama's documented API does not yet expose the website's full per-model history or individual purchased-credit expiries. The dialog links to Ollama Usage for website-only details. Missing fields are explained, never replaced with invented values. Balances follow the configured server account, which can differ from the user's website login.

Monthly balance

Actual included and purchased credits from `/api/balance`.

Rolling history

7/30-day cloud spend and tokens; separate from the billing period.

Local model graphs

Measured Tlamatini main-chat activity, not website-wide history.

Refresh

Gauge events and visible-dialog refresh; cloud cache lasts 60 seconds.

Missing data

Explain unavailable or stale values; no guessed balance or token count.

References

docs.ollama.com/api/balance and [/api/cloud-usage](https://docs.ollama.com/api/cloud-usage); [docs/usage-dashboard.md](https://docs.ollama.com/usage-dashboard.md).

◆ ADAPTIVE LAYOUT

Report what the account supplies

The response shape selects the layout; a plan label is not a credit entitlement.

Free, Pro and Max accounts with monthly balance fields use credit cards. Legacy session and weekly responses use reported remaining percentages and reset times; unsupported cost and token fields are omitted. Purchased-only responses display only the supplied balance. A zero monthly allowance does not produce an invented percentage.

As checked on October 10, 2026, Ollama lists Free, Pro, Max, Team and Enterprise, with no cloud plan named Unlimited. Unlimited local execution is a different claim. Team balances are shared, while this adapter requests default member usage, not administrator team scope. Commercial terms remain at ollama.com/pricing; they are not hardcoded into the dialog.

Monthly

Returned allowance, balance, purchased credits and reset period.

Legacy

Remaining session/weekly percentages; no guessed dollar conversion.

Purchased only

Only the reported purchased balance; no invented included quota.

Free or zero

Report actual data and omit percentages when allowance is zero.

Team

Shared credit balance is distinct from member-scoped usage.

Unlimited label

Never interpreted as unlimited cloud credits.

◆ AGENTIC CONTROL PANEL

Design agent workflows visually

Drag any of the 89 agent types onto the canvas, connect output triangles to input triangles, configure each node, validate, and press Start.

Agents run as separate, log-emitting processes that talk through their `config.yaml` files, with a live LED for each. Flows can be paused, resumed and stopped, and saved or loaded as `.flw` files. FlowCreator can design a flow from a sentence, and FlowHypervisor watches a running flow and raises an alert when something looks wrong.

Since v1.70.0 the canvas is a real editor. Duplicate copies agents together with their saved settings and the connections between them; Undo also restores an agent's configuration. Editing locks while a flow runs, so a flow is never changed underneath its own processes.

Undo and Redo

Up to 1,024 steps, configuration included.

Duplicate

Copies settings and internal connections.

Zoom

25 to 200 percent, and Fit.

Agent search

Filters the 89 agent types by name.

Starters

Locates each Starter on a large canvas.

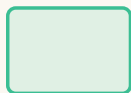
Help

Explains every gesture; • marks unsaved changes.

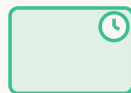
◆ PROMPT FLOW PANEL

Draw a conversation, then press Play

The Prompt Flow Panel turns a chain of prompts into a diagram of seven operations plus static review notes, saved as a portable `.fpmt` file and played against your configured models.

**Prompt**

Sends its text to the model; Multi-Turn and ACPX can be enabled.

**Programmed Prompt**

Waits for a scheduled time or a delay, then sends.

**Decision**

Takes Yes or No from the last answer, or asks you.

**Feed embeddings**

Adds reference text to this run's retrieval context.

**Flush embeddings**

Clears the run's retrieval context.

**Clean History**

Clears the run's conversation and last output.

**User Input**

Stops to ask you something; your reply joins the run.

**User Commentary**

A static speech-bubble note for reviewers; it never runs.

◆ PROMPT FLOW PANEL

Playback that stays in its own lane

Validate checks the diagram, Play runs it, and every run keeps its own conversation, embeddings and cancellation identity, so it never disturbs your chat.

Write `{{last_output}}` to pass the previous answer forward. Decisions compare strings — contains, does not contain, equals, empty, with optional case matching — and never evaluate expressions from a file. Cycles are allowed and bounded by a maximum number of executed operations, 500 by default and up to 5,000.

The running figure lights up and traversed connections are highlighted. Pause lets the current operation finish and holds the next; Stop drains the running worker before another run may start. Opening a file never runs it, and a User Commentary note never runs at all.

Feed embeddings requires a real vector store. A provider error cannot become success through the chat stack's prompt-only fallback, including with tools enabled. The flow reports failure before its next operation and keeps any previously accepted context.

Format

JSON `tlamatini-prompting-flow`, version 2; version 1 files migrate.

Limits

5 MiB, 500 assets, 1,000 connections.

Isolation

Empty history and embeddings at every start.

Validation

Checked in the browser and again on the server.

Not .pmt

Plain-text system prompts keep .pmt and are refused.

Drafts

Kept per user in browser storage.

◆ STATIC REVIEW NOTES

Writing and formatting a commentary

User Commentary is a review note on the canvas. User Input is the operation that pauses playback for a reply, using the notched figure.

Double-click a bubble or press Enter to write directly in it. Select words or paragraphs, then use the floating mini toolbar for font, size, text color, bold, italic and underline. For example, one bubble can have a large Verdana heading, an italic Arial sentence and a Georgia paragraph. With only a caret, formatting applies to subsequent typing. Outside editing, it applies to the whole selected note. Bubble color and alignment affect the note.

Drag any of the eight borders or corners to resize, even while writing. The handles also accept arrow keys. Fit text removes spare height, and the bubble grows to contain all wrapped text. There are no internal scrollbars, numeric size fields or commentary configuration dialogs.

Done or Ctrl+Enter saves the complete edit. Cancel or Escape restores its original text, styles, dimensions and position. Ctrl+Z/Y works inside the editor. Completed edits, moves, resizes and duplicates participate in the diagram's Undo/Redo. Copying between comment editors preserves formatting. External clipboard text stays literal.

◆ PORTABLE FLOW FILES

Comments survive save, open and draft recovery

Version 2 .fpmt files retain independent notes with their full text, formatting and geometry.

Each note stores allowlisted text runs plus matching literal text. Runs carry font, size, text color, bold, italic and underline. Older plain version 2 notes normalize to one run. Version 1 executable commentary migrates to User Input with its identifiers, connections, settings and reply/cancellation behavior intact. Opening a file never starts playback.

A note has no connection ports or Start state and never enters model context, conversation history or playback step counts. Tokens such as `{{last_output}}` and HTML-like text remain literal in a comment. Several independently styled notes can share a diagram.

The per-user browser draft keeps the existing .draft.v1 key for compatibility. Portable files retain the .fpmt extension. The similarly named .pmt files belong to system prompts and remain a separate format. Limits are 5 MiB per flow, 500 assets, 1,000 connections, 100,000 characters and 10,000 formatted runs per note.

◆ OPENING FLOW FILES

The right editor, from chat or Windows

October 4 source changes: .flw opens agents; .fpmt opens prompting flows. Opening never runs them.

The main chat routes both formats into their own editor tabs and keeps its current document. An external file launch validates a bounded snapshot and reuses a matching running server before any database startup work. A cold launch starts the application on its configured port.

Login returns to the requested file. Short-lived, single-use opening links carry a random token rather than a filesystem path; browser uploads are bound to the signed-in user. Invalid formats, unsupported versions and files over 5 MiB are rejected before replacement.

Unsaved diagrams and recovered drafts require confirmation. Rich commentary runs retain their fonts, styles and layout through .fpmt save/open. Agent-flow opening now awaits validation and session preparation; failed credential redaction blocks unsafe .flw downloads.

From chat

Open either flow in its editor and keep the chat document.

From Windows

Start once or reuse the running app; return through login.

Preserve work

Validate before replacement; confirm unsaved changes.

No automatic run

Opening prepares the diagram. Play remains a user action.

◆ WINDOWS FILE TYPES

Register, repair and remove both formats

Per-user associations, installation ownership and Windows default-app choices.

A shared PowerShell helper registers .flw and .fpmt with separate ProgIDs, friendly names, icons, Open With entries and Default Apps capabilities. Status reports the current registration; DefaultApps opens Windows Settings so the user can choose a default.

Installation registers both formats. Update Repair preserves explicit unregistration and never takes another installation's owned entry. Removal deletes only this installation's values, preserving Windows UserChoice and other applications' Open With entries. Historical .flw wrappers cover both formats so existing installer/uninstaller binaries remain compatible.

The scripts and opening modules are included in frozen and self-modify packaging. Native registry checks use an isolated test key, and visible Chrome tests exercise real file opening and login. The October 4 release campaign also rebuilt local main and uninstaller executables. The work was committed to main that day and is carried by the v1.76.0 tag; compiled acceptance is recorded separately from source tests. See docs/windows-flow-files.md for commands, contracts and verification.

Install

Register both types with names, icons and the correct editor destinations.

Administer

Inspect Status, register or remove one type, or open Default Apps.

Update

Repair owned entries without undoing the user's opt-out.

Uninstall

Remove owned registrations; keep other apps and user documents.

◆ WINDOWS LIFECYCLE

Remove the application and its owned workers

Confirmation, installation ownership, retryable errors and preserved user content.

The real installer registration failure came from an omitted InstallDir whose parameter default evaluated before Windows PowerShell populated PSScriptRoot. The wrappers now resolve that default inside the script. Fresh powershell.exe -File tests exercise the same entry point from an unrelated working directory.

The uninstaller first requires the main application to close. After confirmation it stops workers belonging to the selected installation and their descendants. Exact path boundaries, creation times and inherited TLAMATINI_AGENTS_ROOT identify agent children even after reparenting; unrelated processes and Explorer remain running.

A temporary independent copy allows the installed uninstaller itself to be removed. Helpers and the installation marker remain until file and owned-registry removal succeed, so a partial failure can be retried. Locked files and denied registry deletion report failure. Drive roots, shared folders, source checkouts and redirected targets are rejected.

Since v1.76.0, when Windows still holds Uninstaller.exe itself, the file is retried, moved aside and removed by a hidden cleanup after the uninstaller exits, so the uninstall finishes cleanly.

Agents and nonempty user-content directories remain under the existing preservation contract. A full directory/registry reset is a separate authorized action. Verification distinguishes real compiled GUI outcomes from backend tests and records any blocked native launch explicitly.

Installer entry points

Resolve the installation directory inside PowerShell script bodies.

Owned workers

Stop this installation's agents and surviving descendants after confirmation.

Honest retry

Retain removal support until success; report locked files and registry errors.

Preserve user work

Keep agents and nonempty content; notify the shell without restarting Explorer.

◆ LOCAL RELEASE VALIDATION

Repair the boundary, then repeat the real operation

The October 4 repairs, now carried by v1.76.0, have separate source, frozen, package and visual evidence.

The visible campaign checks the actual installer, Windows associations, authenticated file opening, administration, main chat and both flow panels. It also compares several rich comments across repeated file writes and reads, executes local agents, and audits owned workers after shutdown. A configuration dialog opening is not proof that a hardware or external-service agent executed successfully.

System-Metrics and Files-Search now honor the effective configuration and nondefault ports, including UTF-8 BOM files. The path-security and chat-chain readers accept the same encoding without weakening directory boundaries. Both auxiliary context chains use the same effective paths and endpoints; system sockets close after each request and file RPCs have deadlines. Parametrizer restores connections when a saved file contains mappings without explicit connection lists. Model-free agents avoid unrelated model-setting reads. Numeric messaging identifiers keep the same string/integer contract in public and keyed templates.

A rejected embedding request must fail the flow before its next operation. The campaign's provider returned HTTP 401; this is recorded as a blocked capability with a tested failure path. The desktop-control tool also blocked the compiled uninstaller launch. Backend cleanup tests do not certify its unobserved native confirmation and completion screens.

Detailed results and retained failed attempts are in docs/changes/2026-10-04-release-validation.md. The review profile requested by Angela lists each changed file and code segment. The work was committed on October 4 and is carried by the v1.76.0 tag.

Real user paths

Installer, login, Admin, chat, both panels and Windows file opening.

File fidelity

Mixed styles, geometry and rendered text checked across save/open cycles.

Runtime boundaries

Configured MCP endpoints, agent defaults, mappings and identifier types.

Explicit limits

Provider failure and blocked native launch remain visible in the verdict.

◆ RUN OUTPUT

More room for the canvas or the log

Drag the horizontal divider above Run output to give it 5% to 95% of the available pane height.

The percentage uses the combined canvas and output height, excluding headers, the status strip and the divider. The initial allocation is 20%. The canvas and Run output each keep their own scrollbar. Moving the divider preserves canvas zoom, node dimensions and text size. The Operations bar keeps its separate horizontal sizing control.

Tab to the divider and use Up/Down for one percentage point, or Shift+Up/Down for five. Home selects 5% and End selects 95%. Escape ends a drag. Click the Run output heading to collapse it. Reopening restores the chosen height, and resizing the window retains the ratio.

The browser remembers the output height per signed-in user. This layout preference does not modify a flow file, mark the diagram as edited or add an Undo step. Resizing the panel also leaves the run and its output unchanged.

◆ CATALOG OF PROMPTS

Ready-made prompts, grouped by purpose

The Catalog of Prompts opens with 16 sections, each beginning with a guided Step-by-Step wizard and growing from simple to advanced.

Sections: Voice Commands, Getting Started, Files & Search, Run & Execute, Code & Project Generation, Documents & PDF, Images & Vision, Agents & Flows, ACPX, Skills & MCPs, Desktop Automation, Games & 3D, Firmware & IoT, Security & Recon, Messaging & Contacts, Media & Voice, More. VOICE COMMANDS comes first, because speaking is the shortest way to use her: you talk, Whisperer transcribes, and the transcript becomes the prompt. These catalog workflows use the model-mediated wrapped tool and their card-specific modes; they are distinct from the direct microphone button beside Send.

The mode badges on each card are derived from the prompt text itself, and clicking a card sets the toolbar switches to exactly those modes. Every prompt uses one grammar, so it is always clear whose blank is whose.

Mark	Filled by	Meaning
[[...]]	You	A value you provide, collected at the top with a safe default.
{{ ... }}	Tlamatini	A value she fills at run time.
< ... >	Report slot	Where an answer is printed; never an input.

◆ PDF CANVAS

Read a PDF, then hand her the whole document

Open a PDF beside the chat in a vendored Mozilla PDF.js viewer, then use it as context.

Reading uploads nothing and calls no model: the viewer reads byte ranges straight from your disk, so there is no size or page cutoff on Tlamatini's side. Text selection, navigation, zoom, fit, rotation and in-viewer passwords behave as you expect; Copy lifts the text of every page and Save As returns the original bytes.

Use as context asks one question first: Process images? It starts unticked every time, and nothing runs until you press Continue. Unticked, she reads the selectable text only. Ticked, she also renders every page and analyzes every embedded image with Image-Interpreter. Progress is honest, Cancel really cancels, and partial image failures are reported, not hidden. A PDF password is used in memory only: never written to disk, to the context text or to the log.

A second account signed in from the same browser takes over every open tab, while an open chat keeps its first account, so loading is refused. Since v1.76.0 the message names that cause and the fix, reloading with F5, the header comes back and the progress dialog marks the failed step.

◆ CONFIG ▸ MODELS

Every model choice in one place

The Models dialog exposes 38 model, engine and voice settings for the core services and 21 model-backed agents, in six searchable categories.

Categories: Core 7, Vision 7, Speech 7, Workflows 6, Documents 5, Monitoring & messaging 6. Save submits all values, preserves unrelated configuration and downloads nothing. Reconnect the chat to rebuild its clients; agents pick up a choice at their next configuration load. The model list comes from the configured Ollama servers, asked by the backend with their token, so a remote server's models are offered and accepted. Since v1.76.0 Save also opens the Model Brain's Auto-tuning dialog, which tunes each configured model in front of you and names the source of every value.

Agent templates marked "**@config**" follow the global choice, while a literal value in a workflow remains an explicit override. Wrapped chat launches seed the global choices before any explicit tool argument, and the frozen build proves every model loader runs before it packages.



◆ OPERATOR DIALOGS

Everything else, without editing files

Configuration, credentials, databases and updates are handled from the browser.

Access Keys Wizard

One guided place for provider keys; blank fields keep what is set.

Contacts

Names resolve to addresses, phones and @usernames for messaging agents.

DB menu

WAL-safe backup, and a verified database staged for the next start.

External ▸ MCPs

Search the catalog, activate up to five servers, install runtimes.

ACPX-Skills

Browse, enable, diagnose and reload skills.

Check for updates

Stage the latest published release and swap it in safely.

◆ **One dismissal rule everywhere**

Escape means exactly what a dialog's ✕ means, and an outside click never dismisses anything. A downloading updater is the single dialog that refuses to close.

CHAPTER

IV

The Bestiary

89 agent types in twelve families. Each is a plain-Python program you can read, audit and change.

IN THIS CHAPTER

- ◆ 89 agents, twelve families
- ◆ Flow control
- ◆ Routing and logic
- ◆ Orchestration and intelligence
- ◆ Code, files and shell
- ◆ Documents
- ◆ Web, data and APIs
- ◆ DevOps and infrastructure
- ◆ Quality and security
- ◆ Firmware and engines
- ◆ Desktop and capture
- ◆ Voice, audio and vision
- ◆ Messaging and monitoring



◆ THE AGENTS

89 agents, twelve families

Every agent follows one skeleton: a template directory, a `config.yaml`, a session-scoped pool copy, PID and log files, and explicit source and target wiring.

67 of them also run from the chat as wrapped `chat_agent_*` tools, each in its own isolated runtime folder. Every wrapped agent appears in the Exec Report automatically, and each ships at least one example prompt in the catalog.

Display names keep their exact designed casing — STM32er, PDFer, LaTeXer, ACPXer — while directories, pools and style classes use lowercase forms.



◆ AGENT FAMILY · 6 AGENTS

Flow control

Begin, pause, schedule and end a flow.

Starter

The entry point of every flow; launches its targets when you press Start.

Ender

Ends a flow gracefully: kills its targets, then starts backups and cleaners.

Stopper

Watches logs for a pattern and stops exactly those source agents.

Cleaner

Post-run janitor that removes the logs and PID files of finished agents.

Sleeper

Waits a configured number of milliseconds, then starts its targets.

Croner

A daily clock trigger that starts its targets at a configured HH:MM.

◆ AGENT FAMILY · 7 AGENTS

Routing and logic

Decide where the flow goes next.

Raiser

Fires its targets the moment a pattern appears in a watched log.

Forker

Automatic A/B router: whichever of two patterns appears first picks the path.

Asker

Asks you in the chat to choose Path A or Path B: a human in the loop.

Counter

A persistent tally that routes below, or at and above, a threshold.

AND

Two-input gate that fires once both sources have matched.

OR

Two-input gate that fires as soon as either source matches.

Barrier

N-input fan-in that releases once every configured source has arrived.

◆ AGENT FAMILY · 11 AGENTS

Orchestration and intelligence

Design, connect, supervise and reason.

Parametrizer

Maps one agent's structured output into the next agent's configuration.

FlowCreator

Designs a complete, validated .flw workflow from a plain objective.

FlowHypervisor

A model watchdog that reads a running flow: OK or ATTENTION NEEDED.

FlowBacker

Backs up a session's logs and configurations before clean-up.

Gatewayer

Webhook and folder-drop ingress that turns outside events into triggers.

Gateway Relayer

Relays GitHub and GitLab webhooks into Gatewayer.

Node Manager

Infrastructure registry: discovers nodes, probes heartbeats and capabilities.

Prompter

Sends one prompt to a model and logs a structured answer.

Summarizer

Summarizes text once, or watches logs and fires on a detected event.

ACPXer

Runs one external coding-agent CLI turn on the canvas and captures its answer.

MCP Doctor

Diagnoses an external MCP server on paper, before any live connection.

◆ AGENT FAMILY · 13 AGENTS

Code, files and shell

Find, read, change and run.

Executer

Runs a shell command, optionally in its own visible console.

Pythonxer

Runs inline Python behind a compile() and Ruff correctness gate.

Editor

Byte-exact find-and-replace in one file; refuses an ambiguous match.

Grepper

Encoding-aware regex search, plus verbatim reads of a line range.

Globber

Finds files by glob pattern, newest first.

File-Creator

Creates a file with exact content, including a base64 channel.

Mover

Moves or copies files by glob pattern.

Deleter

Deletes by glob inside a guarded working directory.

File-Interpreter

Reads DOCX, PPTX, XLSX, PDF and more, optionally summarizing them.

File-Extractor

Extracts raw text from PDF, DOCX and similar documents.

De-Compressor

Compresses or decompresses .zip, .7z, .gz and tar archives.

J-Decompiler

Decompiles JAR and WAR archives with the bundled jd-cli.

Gitter

Runs git operations and custom git commands.

◆ AGENT FAMILY · 3 AGENTS

Documents

Compose, typeset and present.

PDFer

Composes PDFs from text, Markdown, HTML and images: 24 styles, 20 themes.

PPTXer

Builds editable PowerPoint decks whose text is measured, never truncated.

LaTeXer

Typesets LaTeX projects, with a self-healing eight-rung repair ladder.

◆ AGENT FAMILY · 7 AGENTS

Web, data and APIs

Search, browse, call and measure.

Googler

Resilient two-tier web search with a structured Google-dork builder.

Crawler

Fetches a page raw or as text and analyzes it with a model.

Playwrighter

Drives a real browser through scripted steps: log in, fill, click, extract.

Apirer

Calls REST APIs with any method, headers, body and timeout.

SQLer

Runs Python against Microsoft SQL Server with a ready cursor.

Mongoxer

Runs Python against MongoDB with a ready database handle.

NetSpeed-Calculator

Measures your connection with confidence intervals and bufferbloat.

◆ AGENT FAMILY · 6 AGENTS

DevOps and infrastructure

Containers, clusters, pipelines and hosts.

Dockerer

A Docker and docker-compose front-end.

Kubernetes

A kubectl front-end with a structured, truthful result.

Jenkinser

Triggers Jenkins jobs, handling the CSRF crumb.

SSHer

Runs a command on a remote host over SSH.

SCPer

Sends or receives files over SCP.

PSer

Finds a running process from a fuzzy name with a model's help.

◆ AGENT FAMILY · 8 AGENTS

Quality and security

Review, analyze, assess and protect.

Reviewer

Model-driven review of a git diff with an APPROVE or REQUEST_CHANGES verdict.

Analyzer

Deterministic static analysis: bandit, semgrep, ruff, eslint, gitleaks, pip-audit.

Kalier

Authorized Kali Linux tooling through MCP-Kali-Server.

Discoverer

The ProjectDiscovery recon suite on a private, self-installing Go toolchain.

Nmapper

A local, use-only nmap bridge; unprivileged connect scan by default.

Kyber-KeyGen

Generates CRYSTALS-Kyber post-quantum key pairs.

Kyber-Cipher

Encrypts with Kyber post-quantum cryptography.

Kyber-DeCipher

Decrypts Kyber-protected data.

◆ AGENT FAMILY · 6 AGENTS

Firmware and engines

Real boards and live editors.

STM32er

STM32 firmware from Blue Pill to H7: build, flash, observe, fail-safe.

ESP32er

ESP32 firmware through PlatformIO, from a template that compiles at once.

Arduiner

Arduino firmware through arduino-cli, with cores installed automatically.

ESPHomeer

ESPHome smart-home devices from YAML, no C++ required.

Unrealer

Drives a live Unreal Engine 5 editor: 53 commands in nine categories.

Blenderer

Drives Blender over the official Blender MCP add-on socket.

◆ AGENT FAMILY · 5 AGENTS

Desktop and capture

See the screen, move the pointer, type.

Mouser

Moves and clicks in explicitly declared coordinate spaces.

Keyboarder

Types Unicode into a verified foreground window.

Windower

Focuses, tiles, resizes, lists and closes windows by title.

Shoter

Whole-desktop screenshots with published capture geometry.

Camcorder

Webcam photo or video at the camera's native resolution.

◆ AGENT FAMILY · 7 AGENTS

Voice, audio and vision

Hear, speak, play and look.

Recorder

Microphone capture to WAV at the device's native sample rate.

Whisperer

Speech to text that keeps listening until you stop talking.

Talker

Text to speech, always in Tlamatini's female voice.

AudioPlayer

Plays audio files with volume, truncate and loop control.

VideoPlayer

Plays video with its audio on a chosen display.

Image-Interpreter

Two vision models in parallel, merged by a third into one report.

Video-Analyzer

Rules on robot motion, transcribes audio tracks, or summarizes a video.

◆ AGENT FAMILY · 10 AGENTS

Messaging and monitoring

Reach people and watch systems.

Notifier

Shows an in-browser notification, optionally with a sound.

Emailer

Sends email over SMTP.

RecMailer

Watches an IMAP mailbox for new messages.

Telegrammer

Telegram as the bot or as your own account.

Whatsapper

WhatsApp through Meta's Cloud API, or explicitly from your own number.

Zavuerer

SMS, WhatsApp, Telegram, email and voice through one Zavu key.

TeleTlmatini

Brings the full Tlmatini chat, Multi-Turn included, into Telegram.

Instant Messaging Doctor

Diagnoses and safely repairs Telegram and WhatsApp readiness.

Monitor-Log

Model-powered log monitor.

Monitor Netstat

Model-powered network-port monitor.

CHAPTER

V

Deep Capabilities

The subsystems that give her reach: other agents, other servers, documents, boards, engines and senses.

IN THIS CHAPTER

- ◆ Other coding agents as her children
- ◆ A universal MCP client
- ◆ Procedures, handed over complete
- ◆ Compose, present, typeset
- ◆ A repair ladder that protects your words
- ◆ Real boards, fail-safe by design
- ◆ Unreal Engine and Blender, live
- ◆ Assessment for authorized targets
- ◆ Windows defence, operated by a human
- ◆ She listens until you stop, and answers in her voice
- ◆ Eyes that report their limits
- ◆ Precise hands on a real desktop
- ◆ Three search tiers, one time budget
- ◆ Read the requested page, report what happened
- ◆ Speed measured with its error bar
- ◆ Reaching people deliberately



◆ ACPX

Other coding agents as her children

ACPX spawns external coding-agent CLIs as out-of-process children, talks to them over stdin and stdout, keeps an NDJSON transcript of every turn, and brokers it all to the model as twelve tools.

14 peers are registered: claude, codex, cursor, gemini, qwen, copilot, pi, droid, iflow, kilocode, kimi, kiro, opencode, tlmatini. One-shot peers receive the prompt as a command-line argument and are read to completion; the Tlmatini self-host speaks strict JSON; interactive peers are drained by a transport-aware idle rule. ACPX is a Python port of OpenClaw's ACPX plugin, so its agent identifiers and skill contract are compatible.

ACPX tells the truth about its peers. A version check is not health and exit code 0 is not success: a stdlib-only classifier decides whether a child actually delivered, and a refusal is returned as `ok: false` with a named code instead of a green row. `acp_doctor(deep=True)` sends a real one-line prompt to each peer and names what is broken.

Children are never spawned through the shell, because `cmd.exe` silently cuts a command line at its first newline. `npm` and `pnpm` shims are rewritten to `node.exe` with the script path.

Non-delivery code	Meaning
PERMISSION_BLOCKED	The child stopped at its own permission prompt.
AUTH_FAILED · CONFIG_INVALID	It could not authenticate, or refused its own config.
NO_CREDIT · USAGE_LIMIT	No credit, or a plan or rate limit was reached.
UPSTREAM_ERROR	The model provider returned a server-side error.
NO_OUTPUT · CHILD_ERROR	Nothing legible came back, or it failed without an answer.
WORKSPACE_NOT_TRUSTED	Its working folder is untrusted, so permissions were ignored.

◆ EXTERNAL MCPS

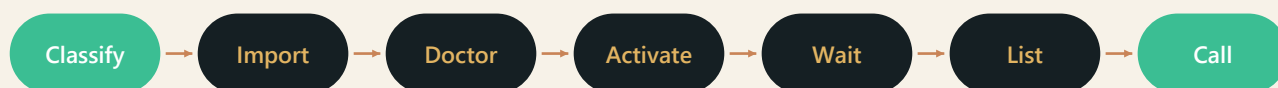
A universal MCP client

Connect Tlamatini to any MCP server declared in a JSON catalog and use its tools at once, with no code written for the server.

The catalog uses the standard `mcpServers` shape and lives beside `config.json`, preserved across updates. Four transports connect — stdio, streamable HTTP, SSE and WebSocket — lazily on a background thread, so a slow server can never delay an answer. Up to five servers are active at once, and each remote tool appears as `ext__<server>__<tool>`.

Most servers start with `npm` or `uvx`, which a fresh machine lacks. A private runtime provisioner downloads Node and uv once from their official sources into the user's profile, verifies checksums, installs atomically, needs no administrator rights and never changes the system PATH.

Memory and Sequential Thinking ship in every installation, inactive. A default you delete is remembered and never resurrected; one you edit is never overwritten.



The adding-external-mcp skill: one guarded onboarding sequence.

◆ SKILLS

Procedures, handed over complete

29 `SKILL.md` packages are loaded at start-up and offered to the model through `list_skills` and `invoke_skill`.

Invoking an in-process skill is a planning handoff, not a hidden execution. It returns the complete procedure, the contract of outputs still owed, and an honest statement of what is enforced — argument checks, secret redaction and time budgets — and what remains declared policy, gated by the normal Configure and Ask Execs surfaces.

Skills include code review, security audit, the Kali pentest runbook, flow making, skill creation, summarization, the adding-external-mcp runbook, Roblox Studio, repository audit and refactor helpers, and integration guides for GitHub, Gmail, Slack, Jira, Notion, Todoist, Trello and weather. The ACPX-Skills menu enables, diagnoses and reloads them without a restart.

◆ Credentials stay inside

An input marked sensitive, or named like a credential, is redacted from every audit event and from the returned result: no prefix, no length, no hash.

◆ THE DOCUMENT FAMILY

Compose, present, typeset

Three agents produce documents, each with its own craft, and none of them calls a file clean just because it was created.

Agent	Craft	Signature features
PDFer	Composes PDFs from text, Markdown, HTML and images	24 visual styles in five families over 20 content themes; measured tables and covers; a layout audit that reopens the file and checks overlap, off-page text, blank pages and contrast.
PPTXer	Builds editable PowerPoint decks	36 named treatments and 17 font pairings; text measured before it is placed, so long content continues onto a new slide; native PowerPoint verification when installed.
LaTeXer	Typesets real LaTeX	Eight templates and 30 signature styles; whole-project builds with bibliography and index convergence; readable diagnostics; an eight-rung repair ladder; a mostly-cut build delivered as <code><name>.DEGRADED.pdf</code> ; MiKTeX recommended.

- ◆ **Created is not verified**
- A saved file only proves it exists. Read PDFer's `layout_clean`, PPTXer's `ground_truth`, or LaTeXer's `status` before calling a document clean.

◆ LATEXER

A repair ladder that protects your words

When a build fails, LaTeXer climbs eight rungs in a fixed order, each tried only if the one before produced no PDF.

Every repair is applied to a copy and re-linted; a repair that makes the lint worse is reverted. The author's file is never overwritten unless `repair_write_back` is set, every rung is recorded in an audit trace, and any block set aside is named with its line number.

Bisect is strictly last because it is the only rung that removes content, and it is skipped when the model rung merely could not be reached. A clean build tells the model plainly that the document is finished, so a good PDF is never "improved" into a broken one.

Since v1.76.0 a PDF is reported only when the delivered file really exists. A build that had to cut a quarter or more of its body is delivered as `<name>.DEGRADED.pdf`, never under the name you asked for, and in a document over 8,000 characters the model rung first repairs only the lines around the compiler's error with a capped request; a reply that never came forbids bisect.

1	lint	Static structural repair before any compiler runs.
2	preamble	Infer packages from the commands actually used.
3	rules	Deterministic rewrites of known-bad constructs.
4	log-directed	Compile, read the real error, fix that line.
5	acquire	Install a package that is genuinely missing.
6	engine swap	Retry with XeLaTeX or LuaLaTeX.
7	model	Ask a model; its answer re-enters at rung 1.
8	bisect	Last: set the failing block aside, and say so.

◆ FIRMWARE

Real boards, fail-safe by design

Four agents take firmware from scaffold to a running board, and each refuses rather than flash something that cannot be right.

Agent	Toolchain	Highlights
STM32er	PlatformIO ststm32 and the STM32 Template Project MCP	Blue Pill through F7, G, L, H7, U5 and WB; zero-config bootstrap; ST-LINK probe checks; newest silicon refused cleanly until its native backend exists.
ESP32er	PlatformIO Core, called directly	Copies a blink-and-print template so a new project compiles at once; serial-aware preflight.
Arduiner	arduino-cli, downloaded on demand	The board is chosen by its FQBN; the board's core is installed automatically.
ESPHomeer	The esphome CLI	Smart-home devices from YAML; headless config generator; USB first flash, OTA after.

◆ Authorized hardware only

Flash, erase, reset and upload change a real attached device. STM32er drives mission-critical robot firmware, which is why its preflight fails safe.

◆ CREATIVE ENGINES

Unreal Engine and Blender, live

Two agents drive running editors over their own sockets, from chat or from the canvas.

Unrealer

Talks to the Unreal MCP plugin over TCP: 53 commands in nine categories — editor, Blueprint, nodes, project, UMG, system, level, asset and material.

Tlmatini's Unreal fork

XaihtUnrealEngineMCP ships the full surface plus a scaffolder for a ready-to-build UE 5.8 C++ project with a Visual Studio 2026 solution.

Blenderer

Speaks the official Blender MCP add-on's code-execution protocol on its local socket, with a rich action catalog so most tasks need no hand-written Python.

Chainable

Both return structured blocks, so Parametrizer can carry a result into the next step and a Forker can branch on it.

◆ SECURITY

Assessment for authorized targets

Offensive and defensive tooling is part of the catalog, always limited to systems you own or are explicitly authorized to test.

Kalier

Kali Linux tools through MCP-Kali-Server: nmap, gobuster, nikto, sqlmap and more.

Discoverer

subfinder, httpx, naabu, katana, nuclei and vulnx on a private Go toolchain.

Nmapper

Uses the nmap you installed; never bundles it; connect scan needs no admin.

Analyzer

bandit, semgrep, ruff, eslint, gitleaks and pip-audit, deterministically.

Reviewer

A senior-engineer review of a diff, with a verdict a flow can branch on.

Kyber

Post-quantum key generation, encryption and decryption.

◆ BLUE-HAT TOOLKIT

Windows defence, operated by a human

The **security/** toolkit is an administrator-run monitoring and response kit, not a chat tool: a person launches it, reads its evidence and decides.

Enablement makes persistent changes. Defender and the firewall keep running, but the installation is excluded from scans, allowed through Controlled Folder Access, and six attack surface reduction rules move to Audit. Those exceptions are real trade-offs, so record a baseline before elevation; there is no automatic rollback.

The monitor watches ten signal families, from Defender health and logons to persistence, ransomware indicators and administrator-group changes. Start with a detect-only baseline, investigate, and arm only when justified. A non-destructive visible harness validates the assets themselves.

Mode	Behaviour
-DetectOnly	Reports what it would block or stop; the safe first run.
default	One armed sweep; may block an attacking IP and stop known attacker tools.
-Watch	Continuous sweeps until Ctrl+C, every 60 seconds by default.
-Aggressive	Also stops dual-use tools outside Tlamatini's paths; incidents only.

◆ VOICE

She listens until you stop, and answers in her voice

Direct chat dictation, workflow Whisperer, Talker and catalog commands have distinct voice contracts.

In standalone/workflow Whisperer, no duration means the gate records while you talk and stops after 3.5 seconds of silence, with a 300-second ceiling. `record_seconds: 0` is the switch; name a duration and it is honoured exactly. The gate starts at a sensitive floor so a speaker who begins at once is heard, and if a sound driver refuses the live stream she records a fixed length and says so. Direct chat dictation instead requires the gate and refuses that fallback.

Transcription runs locally with faster-whisper, on a GPU when present and on the CPU otherwise, or through a cloud Whisper provider. Talker speaks through an Orpheus-compatible model in a female voice — tara, leah, jess, mia or zoe. A male voice is refused by design.

The catalog's first section, VOICE COMMANDS, makes the microphone the keyboard: she transcribes your instruction, reads it back, and only then acts. Anything irreversible still needs your written confirmation under those catalog instructions. Config > Mic selects automatic submission or an editable draft. Actual dispatch uses browser speech for acknowledgment, respecting Silent mode.

◆ VISION AND VIDEO

Eyes that report their limits

Image and video analysis combine independent observers, so one model's guess is never presented as fact.

Image-Interpreter

Two vision models read each image in parallel on their own connections; a third merges both into one definitive report.

Video-Analyzer · robotics

A deterministic motion gate plus two vision models; PASS only when both agree.

Video-Analyzer · transcription

Timestamped speech from chosen audio tracks, locally.

Video-Analyzer · summary

Speech plus sampled visual evidence: scenes, text, facts, decisions and limits.

Capture

Shoter, Camcorder and Recorder save to your known folders with exact names.

Paste to chat

A screenshot pasted into the chat becomes a path she can analyze.

◆ DESKTOP AND BROWSER

Precise hands on a real desktop

Desktop agents state their coordinate spaces and their evidence, because a click that lands in the wrong place is worse than no click.

Mouser

Coordinates are declared: physical screen, desktop, normalized, window or screenshot.

Shoter

Publishes the exact captured rectangle, image size and monitor geometry.

Keyboarder

Binds each keystroke to a verified window and stops on focus loss.

Delivery is not acceptance

`input_sent` proves delivery only; interrupted input is inspected before any replay.

Playwrighter

A real browser through declared steps, with a hold-open knob to watch.

Windower

Focus, move, resize, tile, pin and close windows by title.

◆ WEB SEARCH

Three search tiers, one time budget

The chat search tool and pool agent use the same Googler implementation, with a default 120-second budget and explicit outcomes.

With no engine pinned, six plain-HTTP routes race for relevant results. If none answers, real installed Chrome uses a persistent profile across eight browser routes. Open knowledge sources form the final tier: Wikipedia, arXiv, OpenAlex, Hacker News, GitHub, Internet Archive and Project Gutenberg. The log names the answering engine and tier.

A shared health ledger rests refusing engines for 3, 10 or 30 minutes; an off-topic result is rejected, and a refusing route is not asked again in the same run. Blocked, timed out, unreachable and no matches are separate outcomes. The chat wrapper supports Cancel and enforces a process deadline with a cleanup allowance.

Structured dorks retain presets, grouped filters and `links_only` URL delivery. An explicit engines list skips the HTTP and open-source tiers. Pin Google when Google-only operators matter; fallback engines may broaden their meaning. Licensing and authorization remain yours.

◆ WEB READING

Read the requested page, report what happened

Crawler starts with the supplied page, bounds its work and keeps refusal pages away from the model.

The default `crawl_type` is `page`, with `include_seed` enabled. A range crawl defaults to 25 pages and a 900-second whole-run budget. Each download has a 45-second default wall-clock limit and an 8,000,000-byte cap. These are configurable defaults; zero disables the page count or whole-crawl limit, so unlimited operation must be a deliberate choice.

Headers are read case-insensitively. Bot walls, CAPTCHA pages and access refusals are reported before analysis; timeouts and unreachable pages remain named outcomes. If nothing was analyzed, the final report says so. Crawler reads fetched HTML; it does not execute JavaScript.

Since v1.72.4, both Crawler and Googler use a stack of skipped HTML regions, handle omitted head endings and self-closing tags, and compare the extraction with a plain-text fallback. When that fallback has at least 200 characters and the parser kept less than 25 percent, the fallback takes over. Crawler warns in its log; Googler returns an `extraction_note`. Parser exceptions also use the fallback; healthy extractions stay unchanged.

Page first

Read the seed URL by default; range crawling is explicit.

Budgets

Defaults: 25 pages, 900 seconds overall, 45 seconds per download.

Truthful outcome

Refusals never become source content; no analysis is reported plainly.

Text safety net

Below 25% of a 200+ character fallback: recover and disclose.

◆ NETWORK MEASUREMENT

Speed measured with its error bar

NetSpeed-Calculator publishes a speed with a confidence interval and measures latency under load.

Several keyless providers run in parallel. The measurement discards the TCP slow-start ramp, samples throughput as a derivative, rejects outliers and fuses providers with a random-effects meta-analysis. It grades bufferbloat from A+ to F. A full run transfers about 100 to 200 MB, so it asks before running.

◆ MESSAGING

Reaching people deliberately

Messaging agents use official surfaces by default and make every unofficial route an explicit choice.

Telegrammer

Sends as the bot or as your own account; "send it as me" picks yours.

Whatsapper

Meta's official Cloud API by default; your own number only when you say so.

Zavuerer

One Zavu key for SMS, WhatsApp, Telegram, email and voice, with smart routing.

TeleTlmatini

The full chat in Telegram, on a dedicated account.

Instant Messaging Doctor

Checks tokens, contacts, templates and policy windows.

Contacts

A contact name resolves to the real address when a message is sent.

CHAPTER

VI

Promises She Keeps

Honesty, consent, safety and control, written into the code rather than into a brochure.

IN THIS CHAPTER

- ◆ Truth over comfort
- ◆ Fail open or fail safe, on purpose
- ◆ Your agents, your jurisdiction
- ◆ Your data, handled carefully
- ◆ Nothing runs where nobody can see it



◆ HONESTY

Truth over comfort

Where a system can be confidently wrong, Tlamatini is built to say what she knows and what she does not.

Self-report first

An agent's typed result outranks a one-bit exit code.

Degraded is red

A compromised deliverable is never painted green.

Silence is named

Skipped files, dead transfers and refusals carry a reason.

Recovery is narrated

Retries are shown live and summarized in the answer.

Created is not clean

Documents report their audits before being called finished.

Delivered is not accepted

Desktop input evidence is kept distinct from success.

◆ FAILURE MODES

Fail open or fail safe, on purpose

Every guard chooses its failure direction deliberately, by asking which mistake would hurt you more.

Mechanism	Direction	Why
Binary-content guard	Open	Dropping real context is worse than embedding one odd file.
Verdict parsing	Open	A parse error falls through to the next rule; it never raises.
Port and console settings	Open	A typo must never stop the server from starting.
Runtime provisioner	Open	A failed download never blocks start-up.
Compact-mode fitter	Open	Any doubt sends the complete request, exactly as before.
Ask Execs	Safe	Any doubt resolves to Deny.
Database copies	Safe	An unclear or unchecked copy is reported as a failure.
LaTeXer bisect guard	Safe	When in doubt, protect the author's content.
Firmware preflights	Safe	Refuse rather than build or flash the wrong target.
Deleter	Safe	Refuses protected folders, repository roots and drive roots.

◆ RESPONSIBILITY

Your agents, your jurisdiction

Every agent is a plain-Python program so you can read, audit, edit, restrict or disable it. That transparency is a means of control, not a warranty of safety.

Agents have no independent authority. You decide whether, where, how and with which permissions they run. When you enable, configure, modify, chain or execute an agent, that execution is under your control and your jurisdiction.

You are responsible for reviewing code and configuration, limiting secrets and permissions, authorizing every file, target, browser, shell, API, MCP server, machine and device an agent can reach, supervising its output and complying with the law, policies, licenses and agreements that apply.

By running an agent you accept responsibility for its actions and consequences. Tlamatini's orchestration, documentation and guardrails do not authorize access to third-party systems and cannot replace your own security review, permission controls, monitoring or legal compliance.

◆ DATA SAFETY

Your data, handled carefully

Databases, secrets and scratch files each have one well-defined path.

WAL-safe database

Backups read through the write-ahead log with SQLite's online backup API and are verified with `quick_check`.

Staged replacement

A new database waits in `DB/ToLoad`; the old one is archived with its sidecars before Django starts.

Secrets

Known secret fields are redacted from exported `.flw` files and public builds.

Private contacts

Public builds and source snapshots carry no contact data.

Scratch files

Temporary files stay inside the application's own Temp folder.

Passwords

PDF passwords live in memory only.

◆ TESTED IN THE OPEN

Nothing runs where nobody can see it

Tlamatini's development rule is that every test, build and diagnostic runs visibly, on a real desktop, where its creator can watch each step.

Browser tests run headed in real Chrome, consoles open in the foreground, and whole-desktop photographs are taken by Tlamatini's own Shoter agent — because using the agent is the test of the agent. A stale, transient or timed-out answer is never recorded as a pass.

Source-derived guards stop drift before it ships: the status vocabulary, display names, dialog dismissal, catalog ordering, the Ask Execs allowlist, the self-modify gate and the frozen-bundle module proof are all checked by tests that read the real source.

CHAPTER

VII

Install, Configure, Run

From a fresh Windows machine to a working assistant, and from source to a signed-off release.

IN THIS CHAPTER

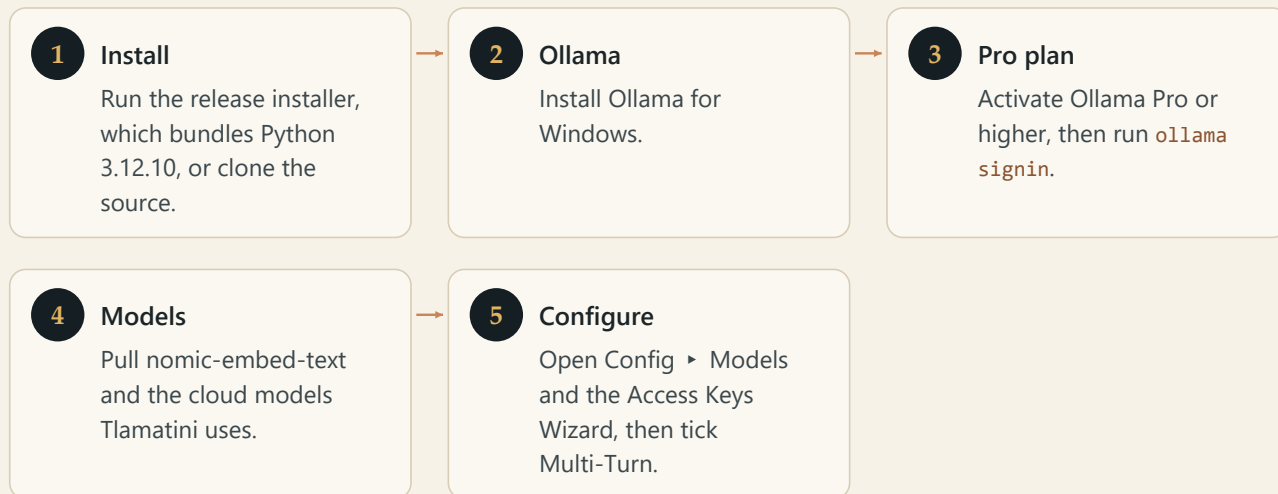
- ◆ Five steps to a working Tlamatini
- ◆ Every referenced Python library is accounted for
- ◆ Two modes and their service ports
- ◆ The settings that matter most
- ◆ From source tree to installer
- ◆ One inventory ships every required helper
- ◆ Packaging evidence and its limits
- ◆ Updating herself without losing you
- ◆ One number, from one source



◆ GET STARTED

Five steps to a working Tlamatini

Tlamatini itself is free. The only purchase is an Ollama plan, bought directly from Ollama.



```
git clone https://github.com/XAIHT/Tlamatini.git
cd Tlamatini
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt
python Tlamatini/manage.py migrate
python Tlamatini/manage.py runserver --noreload
# open http://127.0.0.1:8000/ (default login: user / changeme)
```

◆ DEPENDENCY COVERAGE

Every referenced Python library is accounted for

The October 1 static coverage check includes all 89 agents, build helpers, optional imports and tests.

The refreshed static guard scanned 762 Python files and 69 directly referenced distributions. The main requirements file contains 87 declarations; missing Autobahn, lxml, six, pip and PlatformIO declarations were added. Existing framework and compatibility pins remain.

ESPHome has a separate requirements-esphome.txt manifest because its py7zr and PlatformIO pins conflict with the main environment. Keep it in its private runtime. Blender bpy, Unreal unreal, device toolchains, npm packages and model weights follow their own runtime contracts; they are not missing pip packages.

Coverage guard

scripts/check_requirements_coverage.py checks source and build inventories.

Verification

October 1: no missing declarations, syntax errors or unreviewed dynamic imports.

Earlier evidence

September 27: main/ESPHome dry runs and temporary-environment dependency smoke checks passed.

Limits

Static coverage and selective smoke tests do not certify a clean installation or frozen build.

◆ RUNTIME

Two modes and their service ports

Tlamatini runs either from source or as a frozen executable built by PyInstaller. Only path resolution differs, and `CONFIG_PATH` overrides the configuration location in both.

An installed build resolves `config.json` beside `Tlamatini.exe`; a source run uses `Tlamatini/agent/config.json`. A second, independent axis is self-modification: a build made with `--self-modify` carries her complete, rebuildable source beside the executable, and her self-knowledge travels with it. Without the flag, neither ships. A source run is always self-able, and a toolbar Self-modify switch decides per request whether her self-knowledge is sent; it locks off when the model cannot hold it.

Direct dictation additionally uses a private ephemeral loopback listener for its worker handshake. It closes after authentication; it is not a fourth public service or fixed port.

Both MCP services honor their configured hosts and ports in source and frozen mode; their client URIs must match. Files-Search also honors its worker count and bounds RPC waits. UTF-8 BOM configuration files and `CONFIG_PATH` overrides are supported by both clients.

Port	Protocol	Service
8000 (default)	HTTP + WebSocket	Web interface and chat; change it with <code>django_port</code> .
8765 (default)	WebSocket	System-Metrics MCP; match server port and client URI.
50051 (default)	gRPC	Files-Search MCP; match server port and client URI.

◆ CONFIGURATION

The settings that matter most

A handful of `config.json` keys shape her behaviour; everything else has sensible defaults.

Key	Default	Purpose
<code>django_port</code>	8000	Web port for every launch path; fails open to 8000.
<code>unified_agent_max_iterations</code>	4096	Upper bound on Multi-Turn tool-loop turns.
<code>unified_agent_llm_step_timeout_seconds</code>	900	Watchdog for one model attempt.
<code>llm_client_timeout_seconds</code>	600	How long one model call may take.
<code>model_brain</code>	auto	Tunes each model from formal sources; off restores the legacy sampler.
<code>ollama_repeat_penalty</code>	1.2	Legacy sampler, sent only when <code>model_brain</code> is off.
<code>ollama_num_ctx</code>	1048576	Legacy requested window; the brain clamps it to the real one.
<code>context_sidecar_llm_timeout_seconds</code>	60	Time limit on the context helpers' own model calls.
<code>context_compact_mode</code>	auto	Compact mode locks on for models that cannot hold everything.
<code>binary_context_detection</code>	true	Screens files by content before embedding.
<code>console_quick_edit</code>	false	Keeps a click from pausing a frozen console.
<code>console_colors</code>	true	Colours console lines by level; the log stays plain text.
<code>runtime_autoprovision</code>	true	Lets MCP servers provision Node or uv privately.

◆ Measured, not chosen by taste

The legacy sampler comes from repeated trials on real workloads, and one fast run never justifies a change. The Model Brain now tunes each model from formal sources.

◆ BUILD AND RELEASE

From source tree to installer

Three build scripts produce a standalone Windows release, and the build proves what it ships before it packages anything.



Every stage verifies the one before it.

PyInstaller bundle

Collected static files, templates, agents, skills and helpers, with numpy and OpenCV embedded in both shipped Pythons.

Carriage proof

The build opens the archive it produced and aborts if a required fail-open module is missing.

Runtime gates

Every agent runtime and every model loader is executed inside the new frozen executable, and the Prompt Flow Panel check runs there too.

Integrity receipt

A SHA-256 receipt covers every payload file and is checked again when the installer is assembled.

Size ceiling

The final release ZIP stays under 1,990,000,000 bytes; required files are never dropped to fit.

◆ BUILD REPAIR · SEPTEMBER 27

One inventory ships every required helper

A September 27 repair, carried since v1.72.1, ships the Whisperer settings helper without weakening package verification.

A build correctly refused a runtime receipt because `chat_voice_settings.py` was missing. The verifier required the loose helper, but `build.py` used a separate copy dictionary that omitted it. Compiling `agent.chat_voice_settings` does not satisfy the standalone Whisperer worker running under the carried Python.

`build.py` now calls `copy_root_sources` using the verifier's `ROOT_SOURCES` inventory. All inputs are checked before copying. Every mapped destination is mandatory in both ZIP and extracted-stage receipts, so removing a required file and its receipt entry still fails. The pre-freeze hash baseline continues to reject source changes during a build.

One carrier

Copy and verification share `ROOT_SOURCES`; a new required helper needs no second list.

Worker import

Carry `chat_voice_settings.py` at the installation root as well as compiling the Django module.

Integrity

Reject missing, edited or unmanifested files, including otherwise consistent incomplete receipts.

Rebuild inputs

Mirrored self-update/self-modify sweeps include the shared inventory and report omitted Git checks.

◆ RECORDED VERIFICATION

Packaging evidence and its limits

The September 27 file-level checks establish carriage and integrity; a full installer is a separate verification.

The recorded focused suite ran 146 tests: 145 passed and one Git-dependent census was skipped. It exercised the real carrier with source files, imported the worker from a staged installation, and verified ZIP creation, extraction and receipts with synthetic runtime binaries. Missing or corrupted helpers and extra files were rejected.

The recorded self-update sweep with `--no-git` had no findings. A self-modify snapshot copied 1,579 files with zero copy errors and accounted for all 778 runtime source inputs. These are that audit's dated counts. Neither those checks nor this dossier regeneration ran a complete PyInstaller build, installed a release or performed a live update swap.

Focused suite

146 tests recorded: 145 passed, one skipped; file-only harness exited successfully.

Real files

Every mapped source destination matched; the staged worker resolved its settings helper.

Snapshot

Recorded audit: 1,579 files, 11 redacted, 778 runtime inputs covered, zero copy errors.

Evidence

docs/build-root-assets-verification.md records scope, skipped checks and local transcripts.

◆ SELF-UPDATE

Updating herself without losing you

About ▶ Check for updates fetches the latest published release, stages it, and hands the locked-file swap to an external script.

The swap preserves `config.json`, the database, contacts, the external-MCP catalog, Temp, Templates, generated content and the separately built uninstaller. The database is copied through SQLite's online backup API before shutdown, then new migrations are applied to it on the next launch, so chat history and settings survive while new agents and prompts arrive.

Downloads and staging receipts are verified before shutdown, and a failed prerequisite stops before any file is replaced. While the download runs, the update dialog is sealed and refuses to close.

◆ VERSIONING

One number, from one source

Tlamatini follows Semantic Versioning, with annotated Git tags as the single source of truth.

The runtime resolver, the About window, the `/agent/version/` endpoint and every build artefact report the same base tag. The reported version never carries a development or dirty suffix, and `TLAMATINI_VERSION` can document a release before its tag is cut.

This dossier reports version 1.77.0 and states the exact tag, commit and publication status in its Release chapter.

CHAPTER

VIII

Version and Release Status

Version v1.77.0, the current release, published on GitHub.

IN THIS CHAPTER

- ◆ Usage is verified; rebuild acceptance is next
- ◆ She speaks to you, by name
- ◆ The Model Brain tunes every model
- ◆ Documents and files that tell the truth
- ◆ Smaller fixes that keep the chat moving
- ◆ Graphical notes and resizable Run output
- ◆ Four stop sequences and a clearer Prompt Flow Panel
- ◆ Compact mode and Self-modify become switches
- ◆ Compact mode and readable tables
- ◆ Model lists from the server you configured
- ◆ One card removed from future context
- ◆ Web pages that cannot vanish in silence
- ◆ A context gauge that tells the truth
- ◆ Direct dictation and dependency coverage
- ◆ v1.70.0: the Prompt Flow Panel
- ◆ How the line arrived here
- ◆ Tag, commit and publication, stated plainly

◆ SOURCE CHANGE · OCTOBER 10

Usage is verified; rebuild acceptance is next

The new Usage dialog is an uncommitted source change, not part of the published v1.77.0 binary.

The October 10 source campaign passed 18 Usage tests, 202 existing regressions and 24 visible browser checks, plus source/collected-static byte parity. The live Pro account was compared with Ollama Settings before and after a real model request. Usage opened and refreshed while that request ran. Other-plan, failure and account-switch cases used explicitly synthetic fixtures, not live Free, Max or Unlimited subscriptions.

UsageDaily, migration 0213, records counters by user/model/UTC day. A bounded worker queue keeps inference independent of writes; it is best-effort telemetry, not a billing ledger. `usage_provider.py` and `usage_views.py` provide bounded, authenticated read-only access, credential-safe requests and account-aware stale caching. Build/runtime/source inventories carry the modules, migration, template, JS/CSS and tests together.

Both self-modify and self-update source inclusion sweeps passed. No executable build, installer or real update was run for this feature. An unavailable System-Metrics sidecar in the isolated test does not certify the full product; the real model response and Usage checks completed. The dated test record is in `docs/usage-dashboard.md`. This document refresh does not claim to rerun those browser tests or publish a new version.

220 tests

18 Usage plus 202 existing regressions passed on October 10.

24 browser checks

Visible live Pro comparisons plus clearly synthetic edge cases.

During a request

Usage opened/refreshed while a real model answer completed.

Source carriage

Both inclusion sweeps clean; collected static bytes match source.

Release boundary

Rebuild and installed acceptance remain pending.

Privacy

Account observations and authenticated screenshots stay out of this dossier.

◆ NEW IN v1.77.0

She speaks to you, by name

Tagged on October 9 and published as the current release, so self-update delivers it.

Every fixed line Tlamatini sends to the chat, and that her avatar reads aloud, is now written in the first person and addressed to the user by name: "I'm ready, Angela! You can start chatting with me now." replaces "Your agent is ready." The lines live in one place, `agent/constants.py`; a helper fills in the first name, or the user name when there is no first name, and leaves the name out cleanly when there is neither.

Some of these lines are recognised by their words: the chat page uses them to know it is busy or ready, the avatar to decide what to say, and the Telegram bridge to filter its replies. Every one of those matchers changed in the same commit and still accepts the old wording, so a saved chat history and an older server still read correctly.

The uninstaller no longer erases the user's own work. Uninstalling an installation had deleted the projects kept in `Templates/`, where STM32er, ESP32er, Arduiner, ESPHomer, Unrealr and LaTeXr create their projects by default. It now keeps `application`, `applications`, `content_generated`, `context_files`, `doc_generated` and `Templates` whenever they hold a file, always erases `Temp/`, and still keeps `agents/`. Only a rebuilt Uninstaller.exe carries this: a fresh install or a reinstall copies it, while an in-app update keeps the old one.

Both changes are covered by unit tests (the message tests in `test_compact_mode_switch.py` and `test_uninstaller_mechanics.py`), recorded as passing in the October 9 documentation refresh.

First person

Fixed chat lines say "I" and use your name.

One source

Every fixed line lives in `agent/constants.py`.

Old wording kept

Matchers accept both, so old histories still read.

Templates kept

Uninstalling keeps the projects in `Templates/`.

Temp erased

Scratch goes; folders holding your files stay.

Rebuilt uninstaller

A reinstall carries the fix; an in-app update does not.

◆ NEW IN v1.76.0

The Model Brain tunes every model

Tagged on October 8; the Model Brain commit lands on main one commit after the tag. The published v1.76.0 release was built from main and carries it.

Every model Tlamatini calls now receives its own sampling, thinking level and context size from formal sources, as the How She Works chapter describes. Config ▶ Models ▶ Save opens an Auto-tuning dialog that tunes each configured model in front of you, thinking models keep their own reasoning between tool calls inside one answer, and the stall clocks that threw finished work away are gone.

The console window paints each line by level: errors red, fatal errors white on red, warnings yellow, debug grey and successes green, while `tlamatini.log` stays plain text. At startup she logs which program draws the window, the classic console or Windows Terminal, the default on Windows 11. `console_colors` and the standard `NO_COLOR` variable turn the colours off.

Two smaller repairs ship beside it. A PDF refused because a second account signed in from the same browser now names that cause and the fix, reloading with F5, and the progress dialog marks the failed step. Two chat pages on two different models no longer make each other measure again forever: a model's verdict updates the boxes and stops. On two idle test pages the re-sends fell from 1,081 in one minute to none.

On October 8 the visible Model Brain test passed 25 of 25 checks in a source run and the visible PDF-account test passed 20 of 20. These are dated results, not a claim that this documentation refresh reran them. Publication status is stated in the Release Identity section.

Model Brain

Per-model settings from cited sources; Auto-tuning shows each one.

Reasoning kept

Thinking models keep their train of thought between tool calls.

Coloured console

Lines coloured by level; the log file stays plain text.

Truthful refusal

A PDF refused for a second account says why and how to fix it.

Quiet tabs

Two pages on two models no longer re-measure each other.

Evidence

25 of 25 and 20 of 20 visible checks on October 8.

◆ NEW IN v1.76.0

Documents and files that tell the truth

October 7 and 8: LaTeXer, Mover and the document agents report what really happened.

LaTeXer reports a PDF only when the delivered file really exists and is not empty, and a PDF whose text sticks out more than five points past the margin is reported as created with findings, not as clean. When it had to cut a quarter or more of a broken document's body to produce any PDF at all, it saves the result as `<name>.DEGRADED.pdf` and leaves the requested name untouched. In a document longer than 8,000 characters the model rung first repairs only the lines around the compiler's error with a capped request; an unusable reply falls back to the whole-document request, and a reply that never came forbids the bisect rung.

Mover no longer erases a destination folder to copy into it: folders are merged, each file is staged and published whole, a move removes its source only afterwards, and a failure ends with a failure receipt and a non-zero exit instead of a success. Document and capture agents keep a finished file when Windows Controlled Folder Access blocks the folder, and the security whitelist script now finds every Tlamatini installation and lets its Python save files.

The October 8 LaTeXer change added 37 tests, and all 549 LaTeXer tests passed with it. The October 7 Mover and LaTeXer delivery work passed 126 focused tests.

Real PDFs

Success only when the delivered file exists and is not empty.

Layout findings

Text past the margin is reported, never called clean.

DEGRADED name

A mostly-cut build never wears the name you asked for.

Region repair

Large documents: the lines around the error are repaired first.

Mover

Destinations are merged, never erased; failures are reported.

Protected folders

Finished files survive Controlled Folder Access.

◆ IN v1.76.0 · OCTOBER 5 AND 8

Smaller fixes that keep the chat moving

Committed after the v1.75.0 tag and carried by the v1.76.0 tag.

The Files-Search and System-Metrics helpers ask Ollama their own questions before the main answer. Those calls now have a time limit, `context_sidecar_llm_timeout_seconds`, 60 seconds by default; at the limit, or when you press Cancel, a helper skips its context and the answer goes ahead, so a slow reply can no longer freeze the chat.

When Windows still holds `Uninstaller.exe` while the uninstaller finishes, the file is retried, moved aside and deleted by a hidden cleanup after exit, so the uninstall completes. File-Creator gained `append: true` adds the content to the end of a file, byte for byte, instead of overwriting it.

Context helpers

Their own model calls stop at a time limit or on Cancel.

Uninstaller

A held Uninstaller.exe is moved aside and removed after exit.

File-Creator

`append: true` adds to a file instead of overwriting it.

◆ IN v1.76.0 · OCTOBER 4

Graphical notes and resizable Run output

Committed to main on October 4 and carried by the v1.76.0 tag; local executables were rebuilt to validate them.

User Commentary is edited entirely on the canvas. Double-click or press Enter to write, then format selected words with the floating mini toolbar. A single note can combine different fonts, sizes, text colors, bold, italic and underline. A caret styles the next text you type. Bubble color and alignment apply to the whole note.

Drag any border or corner, including while writing. Fit text removes spare height. Every wrapped line stays inside the bubble without an internal scrollbar. Done or Ctrl+Enter saves the edit. Cancel or Escape restores its text, styles, size and position.

The divider above Run output allocates 5% to 95% of the usable canvas/output height. Only the two viewports change. Canvas zoom, figure dimensions and text sizes stay fixed, and each pane scrolls independently. The browser remembers the ratio per signed-in user.

The October 4 commentary verification passed 29 diagram/runner tests, 3 carriage tests and 23 visible Chrome checkpoints, including three mixed-style comments through three file save/open cycles with identical data and rendered line boxes. The output-divider verification separately passed 14 visible Chrome checkpoints and 3 carriage tests. These are dated results, not a claim that this documentation refresh reran them.

◆ IN v1.76.0 · OCTOBER 3

Four stop sequences and a clearer Prompt Flow Panel

Committed on main after the v1.75.0 tag and carried by the v1.76.0 tag.

Ollama's cloud models refuse a request that carries more than four stop sequences, and the chat model used to send nine, so the chat-history summary, the first call once a conversation grows long, failed the whole request. Every stop list now passes through one fitter that keeps at most four, most important first, and a failed summary no longer fails the answer: the request continues without it, while Cancel still stops it.

The Prompt Flow Panel separates two things that shared one name. User Input is the operation that stops to ask you something; it keeps exactly its old behaviour under a new notched figure. User Commentary is now a static speech-bubble note for reviewers: written in place, coloured and styled freely, growing to hold its whole text without a scrollbar, and never run. New files are saved as version 2; version 1 files open with their old commentary steps turned into User Input, identifiers and connections kept.

The October 3 refresh reran 430 targeted unit tests, the offline Prompt Flow check, ESLint with zero errors, the skills inventory and both inclusion sweeps; all passed, and ruff now reports a clean tree. In a visible Chrome on the real desktop, the commentary test passed all nine checkpoints: long notes without scrollbars, fonts and colours, resize with Undo and Redo, save and reopen, a real User Input run, Escape stopping it, and a version 1 file migrating.

Four stop sequences

Every Ollama request; a failed summary never fails the answer.

User Input

The step that asks you: same mechanism, new figure.

User Commentary

A static review note; it never runs and never reaches the model.

Version 2 files

Version 1 flows migrate on open; identifiers and links are kept.

October 3 evidence

430 unit tests; 9 of 9 visible Chrome checkpoints.

◆ NEW IN v1.75.0

Compact mode and Self-modify become switches

Tagged on October 3: Compact mode is a toolbar switch; the Self-modify switch followed and ships in v1.76.0.

Compact mode as tagged in v1.74.0 was a verdict made behind the user's back. In v1.75.0 it is a toolbar box. Ticking it unticks every row in Configure MCPs, Configure Agents and ACPX-Skills except System-Metrics, Files-Search and Current-Time, and pauses the External MCPs; unticking it ticks every row again. Each row shows the tokens it adds, a model that cannot hold everything turns the box on and locks it, and an answer built from a cut request carries a CONTEXT-WINDOW warning.

Right after the tag, commit 70aeeb87 added the Self-modify box, which ships in v1.76.0. A source run always shows it; a frozen build shows it only when built with --self-modify. On, it sends her self-knowledge, about 28,900 tokens per request; off, one line tells her not to change her own code. A model that cannot hold it shows the box locked off.

The visible Compact-switch run passed 57 of 57 checks on a local qwen2.5 and a free cloud model. The visible Self-modify run passed 28 of 28: with the same 119 tools, the request with the self-knowledge was 117,517 bytes larger than without it. 473 unit tests passed, including a render of the page as a frozen build.

Compact switch

Only the rows you tick; locked on when the model cannot hold everything.

Prices

Every Configure row shows the tokens it adds to each request.

Self-modify

Her self-knowledge on or off; locked off when it does not fit.

Proof

57/57 and 28/28 visible checks; 473 unit tests.

◆ CARRIED FROM v1.74.0

Compact mode and readable tables

Tagged on October 1: every request is fitted to the model it is sent to.

A model that can hold Tlamatini's complete request still receives it byte for byte. A model whose real window is smaller receives Compact mode, described in the How She Works chapter, with a once-per-model dialog, a Compact badge and a locked ACPX box, replaced in v1.75.0 by a switch.

The visible run on a local qwen2.5 exposed three more problems, fixed in the same release. In Compact mode the one-shot file guard now states the real reason a file cannot be opened, and a table the user asked to see is shown as a table instead of fenced code. For every model, an answer-table cell whose text would be unreadable is recoloured.

On October 1 the visible Compact run on the installed qwen2.5:latest passed 37 of 37 checks at about 8,100 tokens per request, with no cuts. On nemotron-3-ultra:cloud every request stayed complete, with Ollama reading 102,539 to 104,259 tokens. A small model can still sometimes re-answer an earlier question before the new one. The v1.74.0 dossier refresh reran the 43 Compact-mode regression tests; all passed.

Fitted

Full when the model can hold the request; Compact when it cannot.

Compact proof

37/37 visible checks on a local qwen2.5, about 8.1K tokens, no cuts.

Full proof

A large cloud model read every request complete: about 103K tokens.

Readable tables

Cells below 3:1 contrast are recoloured, for every model.

◆ CARRIED FROM v1.73.1

Model lists from the server you configured

Tagged on October 1: Config ▶ Models asks the configured Ollama servers, with their token.

Config ▶ Models used to ask the browser's own Ollama for the model list, at an address fixed when the page loaded. After Config ▶ URLs pointed Tlamatini at a remote server, a model installed only there was marked red and could not be saved.

A login-protected endpoint now reads config.json on every call, asks each distinct configured Ollama address in parallel with the same bearer token the chat sends, and merges their model names. Each server reports its own status, and the dialog names the server and the reason when one cannot answer, such as an HTTP 401 that points to the token.

The same release made the start-up GPU step send the token too, so a token-protected remote server no longer answers 401 at every start. A visible run against a simulated token-protected server passed 16 of 16 checks, with every request carrying the token. The v1.74.0 dossier refresh reran the 18 model-catalog regression tests; all passed.

Fresh address

Read from config.json on every call, never fixed at page load.

Token

Every configured server is asked with the chat's own bearer token.

Clear errors

Each server reports its status; a 401 points at the token.

Start-up

The GPU warm-up sends the token too: no 401 at every start.

◆ CARRIED FROM v1.73.0

One card removed from future context

Tagged and published on September 30: Drop throughout chat, with confirmation and history updates.

Saved greetings, user prompts and answers carry their AgentMessage id from the first page render and subsequent WebSocket frames. The drop-message request validates that id against the signed-in conversation owner before deleting rows. A busy guard checks this connection's active run; it is not a global lock across all of the user's tabs.

Successful deletion broadcasts message-dropped with all deleted ids to the user's tabs, then schedules a context-gauge refresh. Dropping a user prompt also removes up to twenty directly following Referenced Rephrase rows, stopping at the first other row. An answer or another user's message is never included in that cleanup.

All 23 focused Drop regression tests passed on September 30, covering ownership, both message roles, neighboring rows, the history window, busy refusal, rephrases and browser/server protocol contracts. The September 30 maintainer record reports 42/42 checks in the extended visible browser harness, including a new real context count after every drop that ends lower than before; this dossier refresh reruns the focused suite without writing to the real memory graph.

Owned rows

Server deletion is scoped to the signed-in user's conversation.

Tabs and gauge

Confirmed ids remove saved cards across tabs and refresh the context estimate.

Rephrases

Prompt cleanup removes following rephrase copies and keeps the answer.

Run guard

An active answer blocks Drop on that connection. Other tabs have separate runs.

◆ CARRIED FROM v1.72.4

Web pages that cannot vanish in silence

Tagged and published on September 29: no web page loses its text without saying so.

Crawler and Googler strip a page to its words by skipping scripts, styles and similar blocks. A block that never closed, such as an omitted closing head tag or a cut-off download, used to swallow the rest of the page. Both now track open blocks with a stack, and a plain-text safety net replaces any result that kept too little, logging a warning when it does.

Two tags travel with it. v1.72.2 added deadline-bound defaults to both agents, and a CAPTCHA or bot wall is reported as blocked, never read as content. v1.72.3 made the chat avatar's lips move clearly while she speaks, never wider than her own portrait.

No silent loss

A stack of open tags replaces a counter in both agents.

Safety net

A plain reading takes over when a parse keeps too little.

Bounded defaults

v1.72.2: search and crawl budgets, cancellation and explicit outcomes.

Refusals named

A CAPTCHA or bot wall comes back as blocked, never as content.

Visible lips

v1.72.3: her lips open and close clearly, without gaping.

◆ CARRIED FROM v1.72.1

A context gauge that tells the truth

Tagged and published on September 28: the ring beside the message box shows Ollama's own count.

The ring now reports the exact request Tlamatini's main chain sends. Bytes are measured; the token figure is Ollama's own `prompt_eval_count` for that request, and the ceiling is the context length Ollama reports. An estimate is shown only until Ollama answers, and says so.

Between questions the ring shows what the next request will cost, recalculated whenever the state changes: page open, Clear history, Drop, Clear context, a context loaded, an answer finished or a mode toggled. It is exact in one-shot and Multi-Turn alike, per connected user, and one-shot questions no longer carry an empty System-Metrics line.

Real tokens

Ollama's `prompt_eval_count`, paired to its own request.

Exact in both modes

Live = at-rest prediction + the question + the Multi-Turn plan.

Main chain only

Out-of-band calls never reach the ring; each user sees their own.

Less waste

The question is sent once; the one-shot placeholder line is gone.

Proof

A visible lab passed 60 of 60 checks against `tlamatini.log`.

◆ CARRIED FROM v1.72.0

Direct dictation and dependency coverage

These September 27 changes were tagged v1.72.0 and published on September 28.

The chat microphone starts Whisperer directly, stops on the configured silence gate, recognizes speech and sends or leaves an editable draft as selected in Config > Mic. The dialog also saves browser-local input, gain, gate and recognition preferences. The worker uses the main application logger and creates no second console. Developer tests remain visibly executed.

The root dependency manifest and separate ESPHome manifest now account for referenced libraries across agents and source. The static guard and detailed dependency audit record coverage and runtime boundaries. They shipped in the published v1.72.0 release; the shared root-asset carrier repair that followed is committed and carried by v1.72.1.

User guide

README.md and BookOfTlmatini.md explain Mic settings, recording, cancellation and both modes.

Runtime contract

docs/chat-microphone-design.md records ownership, protocol, configuration and limits.

Dependencies

docs/dependency-coverage-audit.md records declarations, isolation and verification.

Delivery

Published as v1.72.0 and carried by v1.72.1, with the root-asset carrier repair.

◆ CARRIED RELEASE

v1.70.0: the Prompt Flow Panel

Version 1.70.0 is about designing conversations and editing flows with the same ease.

Prompt Flow Panel

Seven operations, `.fpm` files, validated playback, isolated runs.

Panels and Config

The canvases live under Panels; every Configure dialog under Config.

A real editor

Undo and Redo, Duplicate with settings, zoom, Fit, agent search, Help.

Shared mechanics

Both canvases connect, drag, select and zoom the same way.

Busy-safe menus

Panels stays usable while she answers; one path re-arms the menus.

Context line carried

The context meter, Context Governor and context gauge of v1.65.

◆ LINEAGE

How the line arrived here

Recent tags, oldest first, read from Git.

2026-09-27	◆	v1.71.0	Release v1.71.0 Implementing a fancy fancy microphone to Whsperer in a cero latency direct impl. into Tlamatini's Chat.
2026-09-27	◆	v1.72.0	Implementing a fancy fancy microphone to Whisperer in a cero latency direct implementation into Tlamatini's Chat, and added configuration parameters.
2026-09-28	◆	v1.72.1	Release v1.72.1 Improving gauge meter measurement under one-shot chain.
2026-09-29	◆	v1.72.2	Release v1.72.2 Improving avoidance of blocking/hanging on Googler and Crawler agents.
2026-09-29	◆	v1.72.3	Release v1.72.3 Avatar lips movement improved.
2026-09-29	◆	v1.72.4	Release v1.72.4 Making HTML text extraction unable to lose a page in silence, in Crawler and Googler, by Claude.
2026-09-30	◆	v1.73.0	Release v1.73.0 Implemented the Drop button, which erases one message from the chat AND from the history the model reads, by angelahack1.
2026-10-01	◆	v1.73.1	Release v1.73.1 Making Config > Models ask the configured Ollama with its token, not the browser's local one.
2026-10-01	◆	v1.74.0	Release v1.74.0 Making every request fit the model it is sent to: Compact mode for small models, the complete context for big ones, and readable answer tables.
2026-10-03	◆	v1.75.0	Release v1.75.0 Making Compact mode a real checkbox: it unticks the real Configure rows, locks ON for a small model, prices every row, and warns when a request is cut.
2026-10-08	◆	v1.76.0	Release v1.76.0 Implementin the Model Brain tunning system (comes into action everytime user config any model.
2026-10-09	◆	v1.77.0	Release v1.77.0 Making Tlamatini speak every fixed chat line in the first person and to the user by name, with the word matchers, the avatar and the Telegram bridge updated to recognise both the new and the old wording.

◆ RELEASE IDENTITY

Tag, commit and publication, stated plainly

A tag is not a release, and a commit after a tag is not a new tag. This dossier keeps the three apart.

The annotated tag **v1.77.0** was created on 2026-10-09 and resolves to commit **052bfe0f**, and the remote advertises it.

Since the tag, 9 more commit(s) went into main (**0e6fc72d**, **4535154b**, **60503722**, **2807332a**, **a0338cb8**, **860e1674**, **1692af4b**, **323b694d**, **757a8233**). A build of main still reports version 1.77.0, because the version always carries the tag's number and nothing more.

The newest release published on GitHub is **v1.77.0**, published 2026-10-09.

Fact	Value
Version	1.77.0
Nearest tag	v1.77.0 → 052bfe0f (2026-10-09)
Newest commit on main	0e6fc72d (9 commit(s) after the tag)
Tag on remote	Yes
Latest published release	v1.77.0

CHAPTER

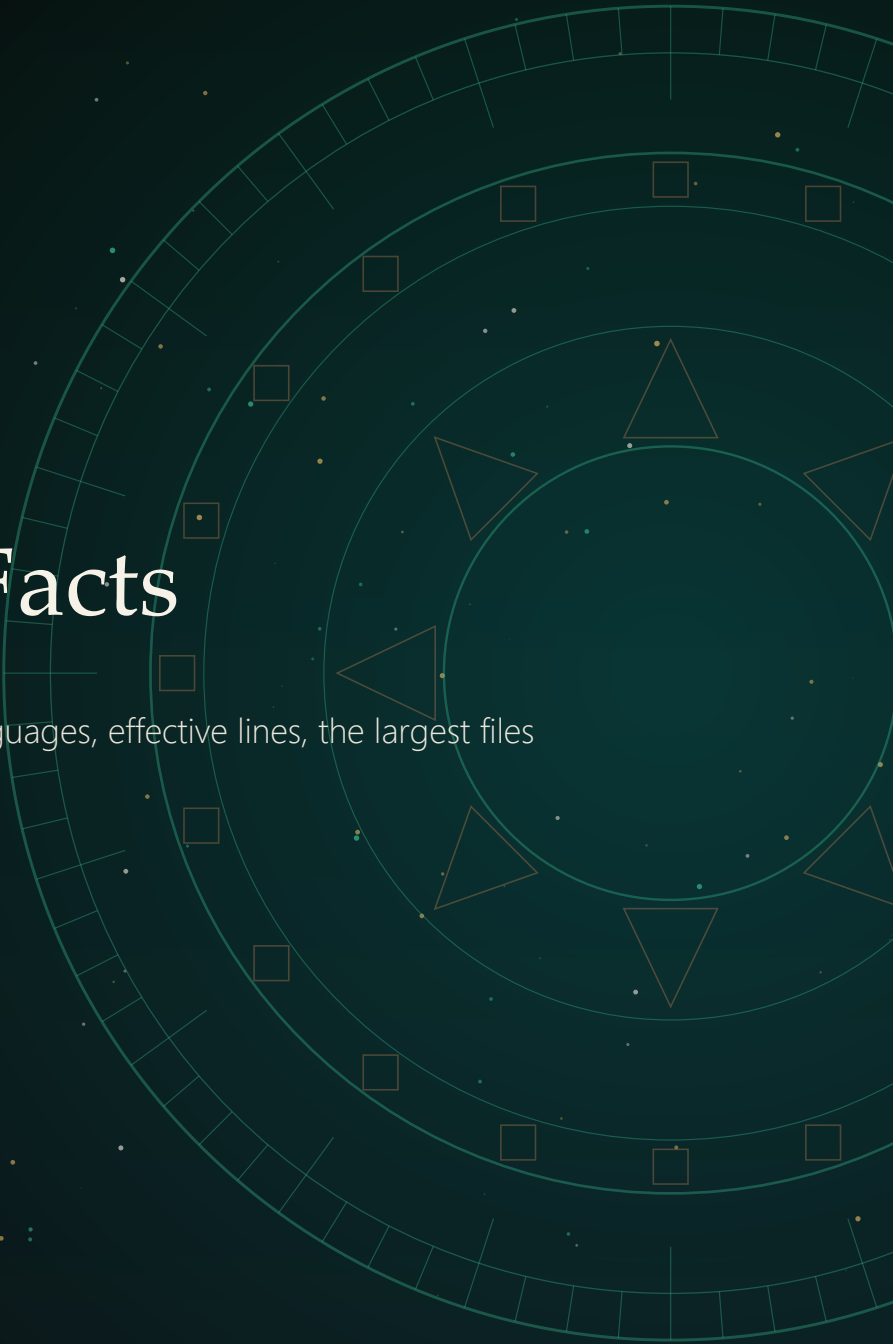
IX

Repository Facts

The source tree measured: files, languages, effective lines, the largest files and recent history.

IN THIS CHAPTER

- ◆ The repository in numbers
- ◆ Effective lines by language
- ◆ Every language, counted
- ◆ Authored and vendored
- ◆ The largest source files
- ◆ Recent commits
- ◆ How these numbers were produced



◆ INVENTORY

The repository in numbers

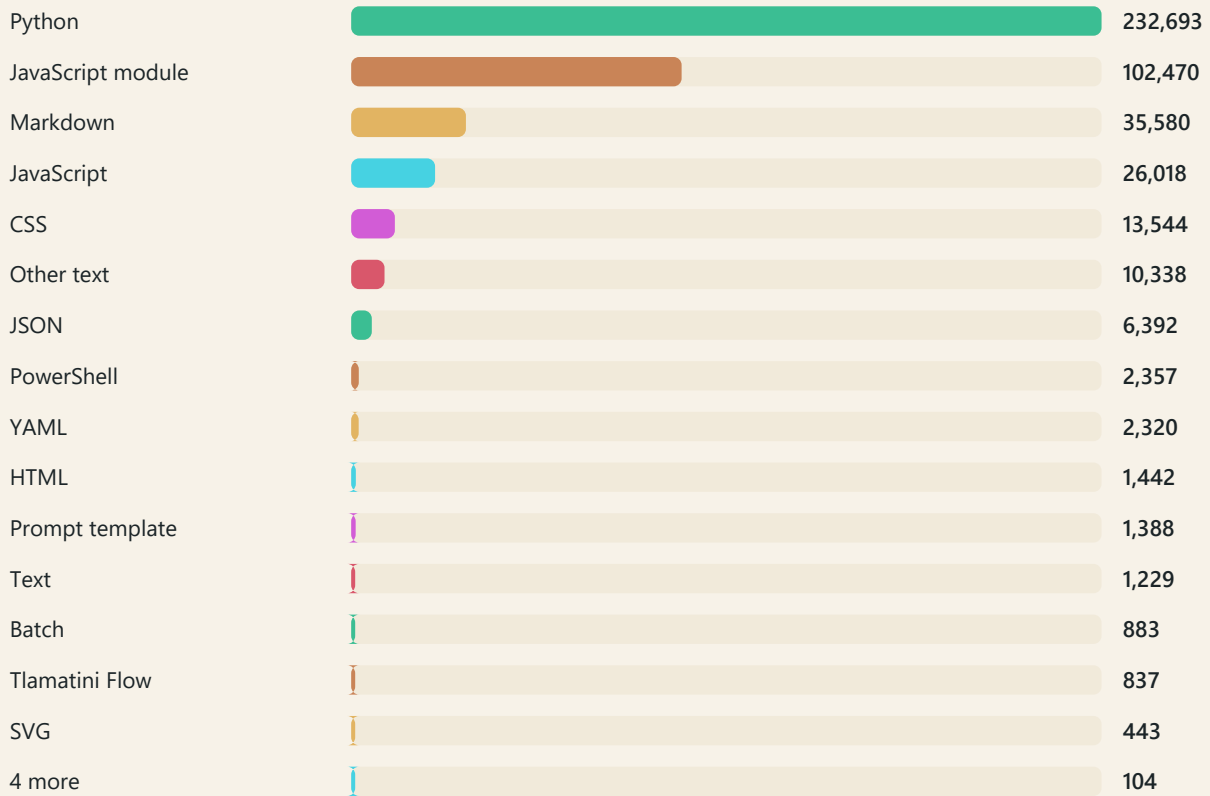
Counts come from Git and from the source files at the moment of generation.

Measure	Value
Inspected commit	0e6fc72d · 2026-10-09 · branch main
Version	1.77.0
Tracked files	1,670
Unignored working-tree additions	13
Text files counted	1,442
Binary assets (counted, not measured)	241
Physical text lines	563,646
Effective lines	438,038
Workflow agent types	89
Wrapped chat-agent tools	67
Built-in Multi-Turn tools	109 = 20 core + 67 wrapped + 12 ACPX/Skill + 10 supervisors
Skills	29
ACPX peers	14
Database migrations	213
JavaScript modules · stylesheets · templates	56 · 20 · 6
Pinned Python requirements	87
Catalog sections	16
Central model settings	38 for 21 model-backed agents

◆ LINE INVENTORY

Effective lines by language

438,038 effective lines across 1,442 text files; 563,646 physical lines.



◆ LINE INVENTORY

Every language, counted

Effective lines exclude blank lines, comment-only lines and Python docstrings.

Language	Files	Physical	Effective	Share
Python	825	328,792	232,693	53.1%
JavaScript module	6	103,847	102,470	23.4%
Markdown	220	46,920	35,580	8.1%
JavaScript	79	32,732	26,018	5.9%
CSS	25	16,882	13,544	3.1%
Other text	42	11,683	10,338	2.4%
JSON	14	6,392	6,392	1.5%
PowerShell	20	3,022	2,357	0.5%
YAML	97	6,278	2,320	0.5%
HTML	7	1,500	1,442	0.3%
Prompt template	2	1,532	1,388	0.3%
Text	10	1,556	1,229	0.3%
Batch	4	1,027	883	0.2%
Tlamatini Flow	4	837	837	0.2%
SVG	82	460	443	0.1%
C++	2	87	50	0.0%
Prompt flow	1	26	26	0.0%
Protocol Buffers	1	33	20	0.0%
INI	1	40	8	0.0%
Total	1,442	563,646	438,038	100.0%

◆ PROVENANCE

Authored and vendored

Third-party frontend libraries are vendored so the application needs no CDN at runtime; their lines are counted separately here so they are never mistaken for Tlamatini's own logic.

Portion	Files	Effective lines
Tlamatini-authored text (application, agents, docs, tests)	1,311	326,991
Vendored frontend libraries (PDF.js, Bootstrap, jQuery ...)	131	111,047
All text files	1,442	438,038

◆ LARGEST FILES

The largest source files

Ranked by effective lines.

File	Language	Effective
Tlmatini/agent/views.py	Python	8,799
Tlmatini/agent/tests.py	Python	5,965
Tlmatini/agent/agents/flowcreator/flow_catalog.json	JSON	5,212
docs/claude/recent-fixes.md	Markdown	3,919
Tlmatini/agent/agents/latexer/latexer.py	Python	3,763
Tlmatini/agent/tools.py	Python	3,373
BookOfTlmatini.md	Markdown	3,191
Tlmatini/agent/mcp_agent.py	Python	2,502
Tlmatini/agent/agents/flowcreator/agent_skill.md	Markdown	2,444
Tlmatini/agent/chat_agent_registry.py	Python	2,164
Tlmatini/agent/external_mcp_manager.py	Python	2,144
Tlmatini/agent/static/agent/css/agent_control_panel.css	CSS	2,133
Tlmatini/agent/doc_generation/dossier_content.py	Python	2,050
Tlmatini/agent/static/agent/js/agent_page_chat.js	JavaScript	2,021
Tlmatini/agent/agents/stm32er/stm32er.py	Python	1,983
Tlmatini/agent/static/agent/css/agent_page.css	CSS	1,974
docs/changes/2026-10-04-commentary-graphical-redesign.patch	Other text	1,908
Tlmatini/agent/consumers.py	Python	1,815

◆ HISTORY

Recent commits

The most recent commits on the inspected branch, newest first.

Commit	Date	Subject
0e6fc72d	2026-10-09	Tlamatini's self-repair Image-Interpreter, by Tlamatini's-AutoBot.
4535154b	2026-10-09	Labelling v1.77.0 as the CURRENT RELEASE on the PDF and PPTX covers, and saying "Git tag" and "version fallback" in CLAUDE.md, GEMINI.md and Tlamatini.md, by Claude
60503722	2026-10-09	Making the PDF and PPTX say plainly that v1.77.0 is the current release, published on GitHub, without the "source tag" wording, and making the native PowerPoint check always measure the deck just written, by Claude
2807332a	2026-10-09	Making the PDF and PPTX dossier state v1.77.0 as the current release, with a TLAMATINI_RELEASE_TAG setting for dossiers made before the release is published, by Claude
a0338cb8	2026-10-09	Pointing the project's Tlamatini MCP server at this repository instead of the old C:\Development\Tlamatini copy, so the mcp_tlamatini_* tools run the definitive code, by Claude
860e1674	2026-10-09	Making Executer report its forked window truthfully: Windows Terminal's hidden helper is no longer mistaken for the console, and the false "another window station" message is gone, by Claude
1692af4b	2026-10-09	Marking v1.77.0 as published on GitHub on 2026-10-09 (marked Latest, delivered by self-update) everywhere in the markdown documentation, with v1.76.0 as the release it replaces, by Claude
323b694d	2026-10-09	Sweeping the source and docs for v1.77.0: regenerating FlowCreator's catalog so it knows File-Creator append, marking the October 4 work as shipped in v1.76.0, and adding the v1.77.0 page to the PDF and PPTX dossier, by Claude
757a8233	2026-10-09	Bumping the markdown documentation to version 1.77.0 (tagged 2026-10-09: first-person chat lines and the uninstaller that keeps Templates/), with v1.76.0 kept as the latest published release until v1.77.0 is published, by Claude
052bfe0f	2026-10-09	Making Tlamatini speak every fixed chat line in the first person and to the user by name, with the word matchers, the avatar and the Telegram bridge updated to recognise both the new and the old wording
a821dc34	2026-10-09	Updating the docs for the published v1.76.0, first-person chat lines and the uninstaller Templates fix, and teaching the self-update sweep that an in-app update keeps the old Uninstaller.exe, by Claude
0520dc97	2026-10-09	Making the uninstaller keep Templates/ and doc_generated/ whenever they hold files, and always erase Temp/, after uninstalling C:\Tlamatini deleted Angela's code in Templates, by Claude

◆ METHOD

How these numbers were produced

The inventory is reproducible: run the generator again and the same tree yields the same counts.

Scope

Every Git-tracked file plus unignored working-tree additions; ignored build output, caches and runtime state are excluded.

Text only

Binary assets such as images, fonts and archives are counted as files, never as lines.

Python

Tokenized: comments, blank lines and module, class and function docstrings are excluded.

Other languages

Block and line comments are stripped, then non-blank lines are counted.

No private values

Configuration values, credentials and addresses are never reproduced.

Regenerate

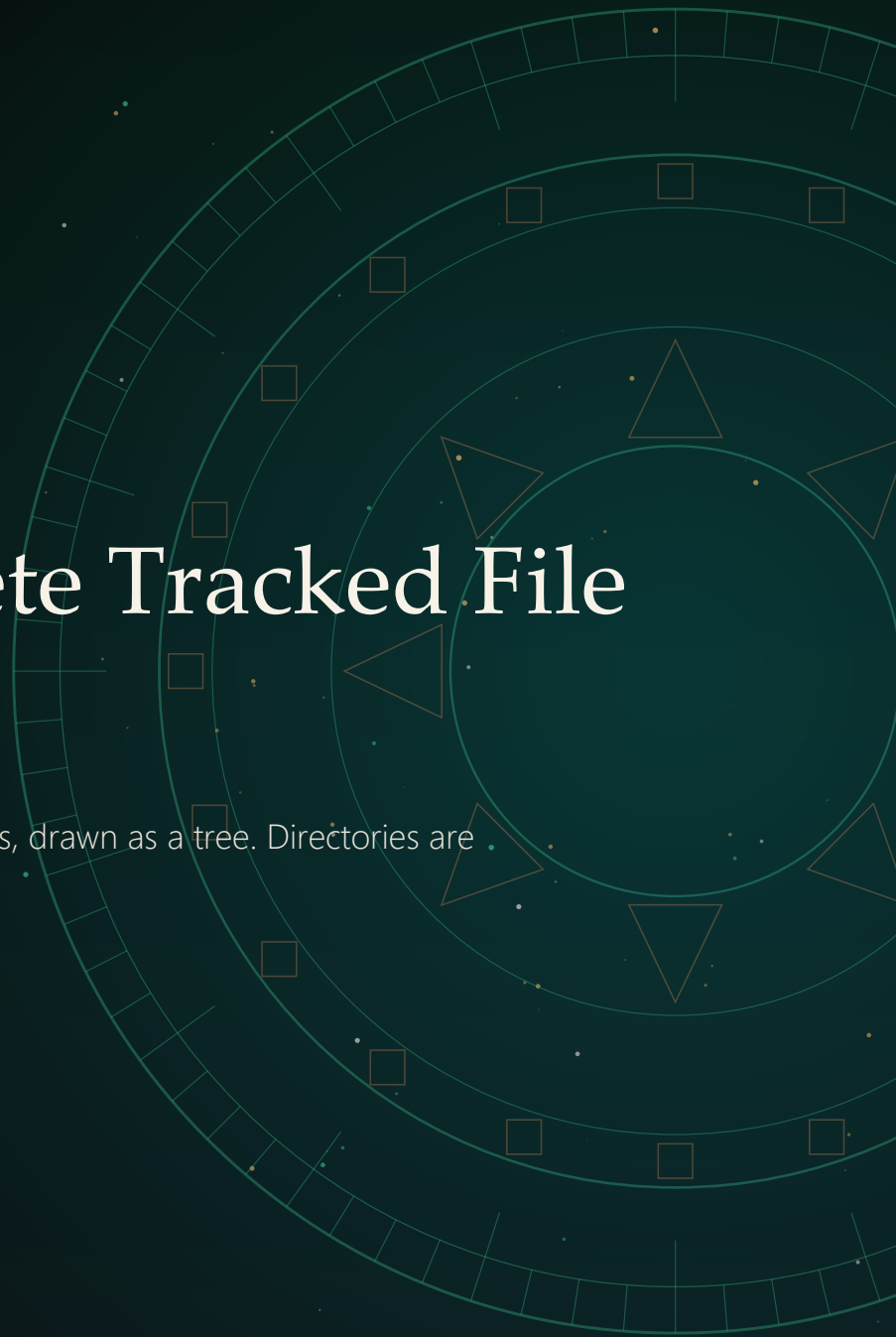
```
python Tlamatini\agent\doc_generation\refresh_
project_docs.py
```

APPENDIX

A

The Complete Tracked File Tree

Every one of the 1,670 files Git tracks, drawn as a tree. Directories are shown in jade.



◆ APPENDIX A

The complete tracked file tree

1,670 tracked files at commit [0e6fc72d](#). Within each directory, files come first and sub-directories follow; a line too long for its column continues on the next line after a » mark.

```

Tlamatini/
├── .gitattributes
├── .gitignore
├── .mcp.json
├── _acpx_pipeline_demo_report.md
├── ACPX.md
├── AGENTS.md
├── agents_descriptions.md
├── apply_update.ps1
├── audit_dependencies.py
├── AvatarImprovementByClaude.md
├── backup_db.py
├── BookOfTlamatini.md
├── build.py
├── build_all.cmd
├── build_complete_private_release.py
├── build_complete_public_release.py
├── build_installer.py
├── build_runtime_assets.py
├── build_uninstaller.py
├── check_private_data.py
├── Claude-Desktop-KALI-MCP-Session.md
├── CLAUDE.md
├── Codex2ClaudeRecommendations.md
├── copy_source_assets.py
├── CreateShortcut.json
├── CreateShortcut.ps1
├── DesignOfIncludingMemoryMCPS.txt
├── Discoverier-new-agent.md
├── eslint.config.mjs
├── example_of_prompt_to_make_de-compresser_agent.txt
├── FirstFinalPlanToSpeedUp.md
├── flow_file_associations.ps1
├── freeingport8000.ps1
├── GEMINI.md
├── git_deny_go.py
├── install.py
├── KIMI.md
├── latest-folding-context-session.md
├── LICENSE
├── MCPMemoriesFlowCreation.flw
├── MessagingConnectorAssistant_Design.md
├── ollama_credits.py
├── OllamaPricing.png
├── OOB_shift_reaper-and-NAMU-Design.pdf
├── package.json
├── PIVOT_CHANGES.md
├── pnpm-lock.yaml
├── preserved_user_state.json
├── private_targets.example.json
├── PromptDesigner.jpg
├── README.md
├── regen_secrets.py
├── register_flw.ps1
├── register_fpmt.ps1
├── RemoveShortcut.ps1
├── requirements-esphome.txt
├── requirements.txt
├── test-reloaded-session.md
├── test_author_banner.py
├── test_check_private_data.py
├── test_perf_3x_visual.py
├── test_private_data_guard.py
├── TestsVisiblesAndVisibleExecutionFromClaude2Codex.md
├── Tlamatini-Kali-Setup.md
├── Tlamatini.ico
├── Tlamatini.jpg
├── Tlamatini.ps1
├── tlamatini_acpx.py
├── tlamatini_app_summary.pdf
├── Tlamatini_eXtended_Artificial_Intelligence_Humanly_Tempered.py
│   └── tx
├── TLAMATINI_MCP.md
├── tlamatini_mcp_server.py
├── uncommit-keys.bat
├── uninstall.py
├── uninstall_processes.py
├── unregister_flw.ps1
├── unregister_fpmt.ps1
├── VERSIONING.md
├── versioning.py
├── .claude/
│   ├── settings.json
│   └── hooks/
│       └── announce_skills.py
│           └── test_announce_skills.py
├── memory/
│   ├── feedback_agent_naming_conventions.md
│   ├── feedback_developer_product_boundary.md
│   ├── feedback_dont_overbuild_exec_safety.md
│   ├── feedback_forbidden_headless_visible_tests.md
│   ├── feedback_foreground_visible_windows.md
│   ├── feedback_hard_real_scenario_tests.md
│   ├── feedback_main_branch_only.md
│   ├── feedback_package_json_version_bump.md
│   ├── feedback_run_tlamatini_agents_visible.md
│   ├── feedback_state_constraints_upfront.md
│   ├── feedback_track_changes_pivot_file.md
│   ├── feedback_update_agent_docs.md
│   ├── feedback_user_owns_git.md
│   ├── MEMORY.md
│   ├── project_acpx_oneshot_prompt.md
│   ├── project_acpx_skills_menu.md
│   ├── project_acpx_toggle_skills.md
│   ├── project_acpxer_added.md
│   ├── project_agent_table_wiped_on_boot.md
│   ├── project_arduiner_agent.md
│   ├── project_ask_execs_feature.md
│   ├── project_audioplayer_agent.md
│   ├── project_build_concurrency_guard.md
│   ├── project_build_tests_and_installer_enterkey.md
│   ├── project_camcorder_agent.md
│   ├── project_carried_python_for_agents.md
│   ├── project_conjunction_parser_fix.md
│   ├── project_daily_chat_test_skill.md
│   ├── project_desktop_ui_lifecycle.md
│   ├── project_director_virtuoso_demo_prompts.md
│   ├── project_doc_refresh_2026_05_06.md
│   ├── project_doc_refresh_2026_05_08.md
│   ├── project_doc_refresh_2026_05_09.md
│   ├── project_eight_fixes_2026_05_07.md
│   ├── project_emailer_recmailers_oneshot.md
│   ├── project_embedding_memory_guard.md
│   ├── project_esp32er_agent.md
│   ├── project_execute_command_bounded_fix.md
│   ├── project_execute_file_foreground_fix.md
│   ├── project_external_exec_safety_layer.md
│   ├── project_flow_compiler_dialog_wins.md
│   ├── project_ghost_distinfo_dependency_audit.md
│   ├── project_kalier_agent.md
│   ├── project_keyboard_wrapped_2026_05_07.md
│   ├── project_latexer_agent.md
│   ├── project_living_canvas_dropped.md
│   ├── project_mouser_wrapped_2026_05_07.md
│   ├── project_native_picker_tcltk_fix.md
│   ├── project_native_toast.md
│   ├── project_netspeed_calculator_agent.md
│   ├── project_numpy_pyinstaller.md
│   ├── project_ollama_source_build_breaks_embeddings.md
│   ├── project_pdfer_agent.md
│   ├── project_planner_followup.md
│   ├── project_playwrighter_agent.md
│   ├── project_playwrighter_hold_open.md
│   ├── project_prompt_catalog_mode_badges.md
│   ├── project_prompt_flow_commentary_input.md
│   ├── project_pythonxer_forkbomb_fix.md
│   ├── project_pythonxer_strict_ruff_gate.md
│   └── project_quote_apostrophe_fix_2026_05_07.md

```

- project_reaper_console_window_fix.md
- project_recorder_agent.md
- project_reviewer_analyzer_agents.md
- project_reviewer_analyzer_demo_prompts.md
- project_reviewer_committed_secrets_falsepos.md
- project_secret_leak_recovery.md
- project_stm32er_agent.md
- project_stm32er_bootstrap_preflight.md
- project_stm32er_demo_prompts.md
- project_stm32er_docs_libs_parametrizer.md
- project_stm32er_hil_serial_proof_fix.md
- project_stm32er_tests.md
- project_summarizer_oneshot_mode.md
- project_taskbar_flash_attention.md
- project_teletlamatini_acpx.md
- project_temp_templates_policy.md
- project_unity_mcp_declined.md
- project_unrealer_expanded_surface.md
- project_versioning_2026_05_15.md
- project_videoplayer_agent.md
- project_windower_agent.md
- user_profile.md

- skills/
 - tlamatini-agent-creation/
 - SKILL.md
 - tlamatini-agent-naming/
 - SKILL.md
 - tlamatini-daily-chat-test/
 - .gitignore
 - SKILL.md
 - harness/
 - _peek.py
 - _stage.ps1
 - acp_backend_banner_test.py
 - acp_backend_banner_test.py.antes_shoter
 - acp_dialog_full_sweep.py
 - acp_dialog_theme_visible.py
 - ask_execs_regression.py
 - ask_execs_regression.py.antes_shoter
 - bulk_toggle_behaviour.py
 - bulk_toggle_visible.py
 - cancel_regression.py
 - cancel_regression.py.antes_shoter
 - compact_mode_visible.py
 - compact_pingpong_visible.py
 - compact_switch_visible.py
 - config.py
 - context_gauge_100.py
 - context_gauge_real_lab.py
 - context_gauge_visible.py
 - context_restore_spinner_visible.py
 - cyber_watchdog_test.py
 - deepseek_remote_chat_visible.py
 - dialog_policy_visible.py
 - dialog_theme_visible.py
 - discoverer_1000.py
 - discoverer_1000.py.antes_shoter
 - drop_message_visible.py
 - flowcreator_wizard_e2e.py
 - full_mode_visible.py
 - googler_dork_hunt.py
 - grepper_lines_visible.py
 - grepper_login_probe.py
 - latexer_openmp_e2e.py
 - latexer_stepbystep_5.py
 - mcp_playwright_suite.py
 - model_brain_visible.py
 - monitor.py
 - ollama_remote_catalog_visible.py
 - pdf_context_visible.py
 - pdfer_nuance_visible.py
 - playwrighter_run.py
 - pptxer_visible.py
 - preflight.py
 - qualify.py
 - questions.py
 - README.md
 - requirements.txt
 - run_test.py
 - self_modify_visible.py
 - shoter_shot.py
 - step_by_step_catalog.py
 - step_by_step_catalog.py.antes_shoter
 - stop_sequence_visible.py
 - talk_test.py
 - uninstaller_visible.py
 - voice_commands_visible.py
 - window-test-screen.py
 - windower_focus.py
 - wrapped_questions.py
 - tlamatini-self-modify-inclusion/
 - SKILL.md
 - scripts/
 - sweep_self_modify.py

- tlamatini-self-update-inclusion/
 - SKILL.md
 - scripts/
 - sweep_self_update.py
- .codex/
 - skills/
 - full-project-pdf-dossier/
 - SKILL.md
 - overlap-safe-pptx-dossier/
 - SKILL.md
 - tlamatini-release-validation/
 - SKILL.md
 - agents/
 - openai.yaml
 - references/
 - acceptance-matrix.md
- .gemin/
 - skills/
 - tlamatini-agent-creation/
 - SKILL.md
 - tlamatini-agent-naming/
 - SKILL.md
 - tlamatini-daily-chat-test/
 - .gitignore
 - SKILL.md
 - harness/
 - acp_backend_banner_test.py
 - acp_backend_banner_test.py.antes_shoter
 - acp_dialog_full_sweep.py
 - acp_dialog_theme_visible.py
 - ask_execs_regression.py
 - ask_execs_regression.py.antes_shoter
 - bulk_toggle_behaviour.py
 - bulk_toggle_visible.py
 - cancel_regression.py
 - cancel_regression.py.antes_shoter
 - config.py
 - cyber_watchdog_test.py
 - dialog_policy_visible.py
 - dialog_theme_visible.py
 - discoverer_1000.py
 - discoverer_1000.py.antes_shoter
 - flowcreator_wizard_e2e.py
 - googler_dork_hunt.py
 - latexer_openmp_e2e.py
 - latexer_stepbystep_5.py
 - mcp_playwright_suite.py
 - playwrighter_run.py
 - qualify.py
 - questions.py
 - README.md
 - requirements.txt
 - run_test.py
 - shoter_shot.py
 - step_by_step_catalog.py
 - step_by_step_catalog.py.antes_shoter
 - talk_test.py
 - wrapped_questions.py
 - tlamatini-self-modify-inclusion/
 - SKILL.md
 - scripts/
 - sweep_self_modify.py
 - tlamatini-self-update-inclusion/
 - SKILL.md
 - scripts/
 - sweep_self_update.py
- .github/
 - workflows/
 - name-guard.yml
- .scanning/
 - finding-policy.json
- AuxTests/
 - mypdf2.py
- demo_flows/
 - demo1_globber_to_parametrizer_to_zavuerer.flw
 - demo2_apirer_to_parametrizer_to_zavuerer.flw
 - demo3_zavuerer_health_to_parametrizer_to_filecreator.flw
- docs/
 - agent-coverage.md
 - build-root-assets-verification.md
 - chat-microphone-design.md
 - companion-app-discovery.md
 - dependency-coverage-audit.md
 - desktop-input-and-flow-contracts.md
 - external_mcp_bulletproof_architecture.md
 - GUI-Manager-design.md
 - model-configuration-verification.md
 - model_configuration.md
 - pdf-canvas.md
 - prompting-flow-designer.md
 - self-management-carriage.md
 - stm32er_all_families_proposal.md
 - visual-analysis-errors.md
 - windows-flow-files.md

```

changes/
├── 2026-10-03-prompt-commentary-input.md
├── 2026-10-03-prompt-commentary-input.patch
├── 2026-10-03-prompt-flow-documentation.md
├── 2026-10-03-prompt-flow-documentation.patch
├── 2026-10-04-commentary-graphical-redesign.md
├── 2026-10-04-commentary-graphical-redesign.patch
├── 2026-10-04-prompt-flow-documentation.md
├── 2026-10-04-release-validation.md
├── 2026-10-04-run-output-resize.md
├── 2026-10-04-run-output-resize.patch
├── 2026-10-04-windows-flow-files.md
├── 2026-10-07-latexer-verification.md
├── 2026-10-07-pdf-mover-incident.md
├── 2026-10-07-windows-access-verification.md
└── claude/
    ├── acpx.md
    ├── agents.md
    ├── architecture.md
    ├── exec-report.md
    ├── frontend.md
    ├── gotchas.md
    ├── INDEX.md
    ├── mcp-tools.md
    ├── multi-turn.md
    └── recent-fixes.md
└── examples/
    └── prompting-kickoff.fpm

pyinstaller_hooks/
├── hook-numpy.py
├── hook-pymupdf.py
└── hook-torch.py

scripts/
├── acp_editor_visible.py
├── chat_microphone_visible.py
├── check_js_parse.mjs
├── check_requirements_coverage.py
├── dossier_visual_walkthrough_visible.py
├── flow_canvas_mechanics_visible.py
├── flow_file_associations_test.ps1
├── flow_files_visible.py
├── installed_panels_visible.py
├── menu_browser_checks.py
├── menu_live_checks.py
├── menu_reference_graph.mjs
├── panel_search_title_visible.py
├── pptxer_visibility_gallery.py
├── prompt_flow_commentary_visible.py
├── prompt_flow_connections_visible.py
├── prompt_flow_extension_visible.py
├── prompt_flow_output_resize_visible.py
├── prompt_flow_selection_visible.py
├── release_acp_editor_checks.py
├── release_acp_runtime_checks.py
├── release_agent_catalog_checks.py
├── release_auth_checks.py
├── release_commentary_checks.py
├── release_extended_checks.py
├── release_local_agent_checks.py
├── release_mcp_checks.py
├── release_output_resize_checks.py
├── release_prompt_runtime_checks.py
├── release_shell_file_checks.py
├── run_menu_state_checks.py
├── skill_inventory.py
├── sync_assistant_skills.py
├── tlamatini_clean_install.ps1
├── update_flow_catalog.py
├── vendor_frontend.py
├── vendor_pdfjs.py
├── verify_latexer_styles.py
├── verify_pdfer_styles.py
├── verify_pptxer_layout.py
├── verify_pptxer_styles.py
└── visual_fatal_dialog_visible.py

security/
├── automated_tests_of_security_assets.py
├── enable_tlamatini_v2.bat
├── README.md
├── run_defender.bat
├── sync_enable_launcher.py
├── test_security_syntax.ps1
├── test_windows_access.ps1
├── tlamatini_defender.ps1
├── tlamatini_whitelist_v2.ps1
├── verify_agent_execution.py
└── windows_access_helpers.ps1

Tests/
├── run_avatar_tests.py
├── run_presence_tests.ps1
├── test_about_window_name_visible.py
├── test_avatar_animation.cjs
├── test_avatar_collectstatic.py
├── test_avatar_presence_visible.py
├── test_avatar_visible.cjs
├── test_pdf_canvas_assets.py
├── test_pdf_canvas_browser.py
├── test_perf_3x_visual.py
├── test_requirements_coverage.py
├── avatar_dev_server/
│   ├── dev_settings.py
│   └── development_test.py
├── Tlamatini/
├── cat_art.py
├── manage.py
├── .agents/
│   └── workflows/
│       └── create_new_agent.md
├── .mcp/
│   └── create_new_mcp.md
├── .skills/
│   └── create_new_skill.md
├── agent/
│   ├── __init__.py
│   ├── access_key_wizard.py
│   ├── admin.py
│   ├── agent_manifest.py
│   ├── agent_verdict.py
│   ├── apps.py
│   ├── cancellation.py
│   ├── capability_registry.py
│   ├── chain_files_search_lcel.py
│   ├── chain_system_lcel.py
│   ├── chat_agent_registry.py
│   ├── chat_agent_runtime.py
│   ├── chat_history_loader.py
│   ├── chat_voice_consumer.py
│   ├── chat_voice_runtime.py
│   ├── chat_voice_settings.py
│   ├── command_togen_pb2.txt
│   ├── command_watchdog.py
│   ├── compact_mode.py
│   ├── config.json
│   ├── config_loader.py
│   ├── constants.py
│   ├── consumers.py
│   ├── contacts.py
│   ├── context_baseline.py
│   ├── context_fitter.py
│   ├── context_governor.py
│   ├── context_sidecar_timeout.py
│   ├── embedding_memory_guard.py
│   ├── exec_permission.py
│   ├── external_mcp_defaults.py
│   ├── external_mcp_manager.py
│   ├── external_mcp.json
│   ├── filesearch.proto
│   ├── filesearch_pb2.py
│   ├── filesearch_pb2_grpc.py
│   ├── flow_file_open.py
│   ├── flow_file_views.py
│   ├── global_execution_planner.py
│   ├── global_state.py
│   ├── gpu_perf.py
│   ├── inet_determiner.py
│   ├── llm_timing.py
│   ├── local_toolcall_parser.py
│   ├── log_identity.py
│   ├── mcp_agent.py
│   ├── mcp_files_search_client.py
│   ├── mcp_files_search_server.py
│   ├── mcp_system_client.py
│   ├── mcp_system_server.py
│   ├── model_brain.py
│   ├── model_brain_chat.py
│   ├── model_brain_views.py
│   ├── model_profiles.json
│   ├── models.py
│   ├── native_dialogs.py
│   ├── ollama_stop.py
│   ├── orphan_reaper.py
│   ├── path_guard.py
│   ├── pdf_context.py
│   ├── pdf_context_jobs.py
│   ├── pdf_context_views.py
│   ├── pdf_image_analysis.py
│   ├── prompt.pmt
│   ├── prompt_flow_panel_consumer.py
│   ├── prompt_flow_panel_runtime.py
│   ├── rag_enhancements.py
│   ├── routing.py
│   ├── run_db_wal_tests.ps1
│   ├── runtime_provisioner.py
│   ├── self_healing.py
│   ├── self_update.py
│   ├── sqlite_copy.py
│   ├── test_access_key_wizard.py
│   └── test_agent_display_names.py

```



```

— test_agent_manifest.py
— test_agent_verdict.py
— test_agents_see_hidden_dirs.py
— test_arduinoer_agent.py
— test_ask_execs_allowlist.py
— test_audioplayer_agent.py
— test_binary_guard.py
— test_blenderer_agent.py
— test_build_pip_quiet.py
— test_build_root_assets.py
— test_build_scripts.py
— test_camcorder_agent.py
— test_cancellation.py
— test_chain_readiness.py
— test_chat_agent_run_wait.py
— test_chat_bridge_bots.py
— test_chat_history_window.py
— test_chat_voice.py
— test_checkbox_bulk_toggle.py
— test_command_watchdog.py
— test_compact_mode_pingpong.py
— test_compact_mode_switch.py
— test_console_colors.py
— test_console_shield.py
— test_contacts.py
— test_context_governor.py
— test_context_real_tokens.py
— test_context_restore_spinner.py
— test_context_sidecar_timeout.py
— test_crawler_agent.py
— test_crawler_resilience.py
— test_createsuperuser_wizard.py
— test_ctrl_c_shutdown.py
— test_db_backup_restore_wal.py
— test_db_backup_safety.py
— test_de_compressor.py
— test_deep_research_prompt.py
— test_deleter_safety.py
— test_desktop_input_agents.py
— test_dialog_dismissal_policy.py
— test_dialog_theme.py
— test_discoverer_agent.py
— test_discoverer_thousand.py
— test_django_port_config.py
— test_drop_message.py
— test_editor_agent.py
— test_embedding_memory_guard.py
— test_esp32er_agent.py
— test_esphomer_agent.py
— test_exec_report_verdict.py
— test_execute_command_bounded.py
— test_executer_window_report.py
— test_external_mcp_add_flow.py
— test_external_mcp_e2e.py
— test_external_mcp_transports.py
— test_external_mcp_universal.py
— test_file_creator_append.py
— test_file_extractor_reader.py
— test_flow_contracts.py
— test_flow_file_open.py
— test_flow_file_views.py
— test_flow_knowledge.py
— test_frontend_mutable_state.py
— test_gitter_custom_command.py
— test_globber_agent.py
— test_googler_agent.py
— test_googler_dorks.py
— test_googler_resilience.py
— test_grepper_agent.py
— test_grepper_encodings.py
— test_grepper_lines_mode.py
— test_image_interpreter_agent.py
— test_instant_messaging_doctor.py
— test_j_decompiler_agent.py
— test_js_gates.py
— test_kalier_agent.py
— test_latexer_agent.py
— test_latexer_delivery_truth.py
— test_latexer_layout_truth.py
— test_latexer_major_removal_and_window_repair.py
— test_latexer_model_repair_truth.py
— test_latexer_repair_ladder.py
— test_latexer_styles.py
— test_latexer_suite.py
— test_latexer_verbatim_channel.py
— test_log_identity.py
— test_mcp_configured_endpoints.py
— test_model_brain.py
— test_model_capacity.py
— test_model_settings.py
— test_mover_file_safety.py
— test_native_dialogs.py
— test_netspeed_calculator_agent.py
— test_nmapper_agent.py

```

```

— test_no_audio_in_tests.py
— test_ollama_catalog_from_server.py
— test_ollama_stop_limit.py
— test_orphan_reaper.py
— test_parametrizer_mcp_doctor.py
— test_password_quoting.py
— test_pdf_context.py
— test_pdfer_agent.py
— test_pdfer_nuance_layout.py
— test_pdfer_styles.py
— test_playwrighter_agent.py
— test_pptxer_agent.py
— test_pptxer_styles.py
— test_pptxer_visibility.py
— test_preserved_user_state.py
— test_program_name_collisions.py
— test_prompt_catalog_contiguous.py
— test_prompt_flow_panel.py
— test_prompt_flow_panel_carriage.py
— test_prompt_flow_panel_runtime.py
— test_prompt_flow_panel_websocket.py
— test_prompt_shape_sentences.py
— test_public_release_targets.py
— test_recorder_agent.py
— test_runtime_provisioner.py
— test_security_assets_carriage.py
— test_self_healing.py
— test_self_knowledge_is_current.py
— test_self_modify_gate.py
— test_self_modify_switch.py
— test_self_update.py
— test_shoter_takes_screenshots.py
— test_sqlite_wal_mode.py
— test_status_vocabulary.py
— test_step_by_step_mode.py
— test_stm32er_agent.py
— test_talker_agent.py
— test_telegrammer_identity.py
— test_temp_dir_policy.py
— test_tool_failure_recovery.py
— test_uninstall_processes.py
— test_uninstaller_lifecycle.py
— test_uninstaller_mechanics.py
— test_unrealer_agent.py
— test_version_guard.py
— test_video_analyzer_agent.py
— test_video_analyzer_content.py
— test_videoplayer_agent.py
— test_visual_fatal_errors.py
— test_voice_commands_catalog.py
— test_watchdog_foreground_exemption.py
— test_web_process_stays_lean.py
— test_whatssapper_identity.py
— test_whisperer_agent.py
— test_win_shim.py
— test_windower_agent.py
— test_windows_app_registration.py
— test_zavuerer_agent.py
— tests.py
— tests_perf_3x.py
— Tlmatini.md
— tools.py
— urls.py
— version.py
— views.py
— visual_error_reporting.py
— web_search_llm.py
— win_shim.py
— window_flash.py
— windows_app_registration.py
— acpx/
  — __init__.py
  — agent_registry.py
  — child_health.py
  — config.py
  — permissions.py
  — runtime.py
  — service.py
  — session_store.py
  — tests.py
  — tools.py
  — windows_spawn.py
— agents/
  — model_settings.py
  — visual_errors.py
  — acpxer/
    — acpxer.py
    — config.yaml
  — analyzer/
    — analyzer.py
    — config.yaml
— and/
  — and.py
  — config.yaml

```


- config.yaml - monitor_netstat.py - mouser/ - config.yaml - mouser.py - mouser_coordinates.py - mover/ - config.yaml - mover.py - netspeed_calculator/ - config.yaml - netspeed_calculator.py - nmapper/ - config.yaml - nmapper.py - node_manager/ - config.yaml - node_manager.md - node_manager.py - nodes_example.json - nodes_example.yaml - notifier/ - config.yaml - notifier.py - or/ - config.yaml - or.py - parametrizer/ - config.yaml - parametrizer.py - pdfer/ - config.yaml - pdfer.py - pdfer_artwork.py - pdfer_atelier.py - pdfer_audit.py - pdfer_color.py - pdfer_consult.py - pdfer_docmodel.py - pdfer_nuance.py - pdfer_ornament.py - pdfer_styles.py - pdfer_tables.py - pdfer_theme.py - pdfer_typography.py - STYLES.md - playwrighter/ - config.yaml - playwrighter.py - pptxer/ - config.yaml - pptxer.py - pptxer_audit.py - pptxer_build.py - pptxer_color.py - pptxer_diagram.py - pptxer_docmodel.py - pptxer_draw.py - pptxer_fonts.py - pptxer_layout.py - pptxer_media.py - pptxer_nuance.py - pptxer_render.py - pptxer_theme.py - STYLES.md - prompter/ - config.yaml - prompter.py - psr/ - config.yaml - psr.py - pythonxer/ - config.yaml - pythonxer.py - raiser/ - config.yaml - raiser.py - recmailer/ - config.yaml - recmailer.py - recorder/ - config.yaml - recorder.py - reviewer/ - config.yaml - reviewer.py - scper/ - config.yaml - scper.py - shoter/ - config.yaml - shoter.py - sleeper/ - config.yaml	- sleeper.py - sqler/ - config.yaml - sqler.py - ssher/ - config.yaml - ssher.py - starter/ - config.yaml - starter.py - stm32er/ - config.yaml - stm32er.py - stopper/ - config.yaml - stopper.py - summarizer/ - config.yaml - summarizer.py - talker/ - config.yaml - talker.py - telegrammer/ - config.yaml - HOW_TO_GET_YOUR_TELEGRAM_ASSETS.md - telegrammer.py - guide_assets/ - telegram_01_botfather_search.jpg - telegram_02_newbot_token.jpg - telegram_03_start_bot_getupdates.jpg - telegram_04_tlamatini_contacts.jpg - teletlamatini/ - config.yaml - teletlamatini.py - unrealer/ - config.yaml - unrealer.py - video_analyzer/ - config.yaml - README.md - video_analyzer.py - video_content.py - videoplayer/ - config.yaml - videoplayer.py - whatsapper/ - config.yaml - HOW_TO_GET_YOUR_WHATSAPP_ASSETS.md - whatsapper.py - guide_assets/ - whatsapp_01_meta_create_app.jpg - whatsapp_02_api_setup_keys.jpg - whatsapp_03_test_recipient_send.jpg - whatsapp_04_system_user_token.jpg - whatsapp_05_tlamatini_contacts.jpg - whisperer/ - chat_worker.py - config.yaml - README.md - whisperer.py - windower/ - config.yaml - windower.py - zavuerer/ - config.yaml - zavuerer.py - doc_generation/ - complete_project_docs.py - dossier_content.py - dossier_pdf.py - dossier_pptx.py - dossier_theme.py - dossier_verify.py - markdown_to_pdf.py - refresh_project_docs.py - test_output.pdf - images/ - ACPXKeysConfigureWizard.jpg - ConfigureModels.jpg - MenuConfig.jpg - mockup.jpg - XAIHT-Tlamatini.mp4 - imaging/ - converter.py - image_interpreter.py - screenshot.py - management/ - init_.py - commands/ - acpx_lint.py - check_agent_runtimes.py - check_prompt_flow_panel.py - startserver.py - migrations/
---	---

0001_initial.py	0095_add_windower_playwrighter_demo_prompts.py
0002_populate_db.py	0096_add_director_virtuoso_demo_prompts.py
0003_add_cleaner.py	0097_add_kalier.py
0004_add_deleter.py	0098_add_chat_agent_kalier_tool.py
0005_add_executer.py	0099_add_kalier_demo_prompts.py
0006_add_notifier.py	0100_add_unrealer_extended_demo_prompts.py
0007_add_stopper.py	0101_add_stm32er.py
0008_add_recmailer.py	0102_add_chat_agent_stm32er_tool.py
0009_add_whatsapp.py	0103_add_stm32er_demo_prompts.py
0010_add_pythoner.py	0104_fix_stm32er_hil_serial_proof.py
0011_add_asker.py	0105_add_esp32er.py
0012_repopulate_all_agents.py	0106_add_chat_agent_esp32er_tool.py
0013_add_forker.py	0107_add_esp32er_demo_prompts.py
0014_repopulate_all_agents.py	0108_add_flow_making_demo_prompt.py
0015_add_shoter.py	0109_add_arduinoer.py
0016_repopulate_all_agents.py	0110_add_chat_agent_arduinoer_tool.py
0017_add_ssh.py	0111_add_arduinoer_demo_prompts.py
0018_repopulate_all_agents.py	0112_add_camcorder.py
0019_add_scper.py	0113_add_chat_agent_camcorder_tool.py
0020_repopulate_all_agents.py	0114_add_recorder.py
0021_add_telegramrx.py	0115_add_chat_agent_recorder_tool.py
0022_repopulate_all_agents.py	0116_add_audioplayer.py
0023_add_mongoxer.py	0117_add_chat_agent_audioplayer_tool.py
0024_repopulate_all_agents.py	0118_add_videoplayer.py
0025_add_telegramer.py	0119_add_chat_agent_videoplayer_tool.py
0026_repopulate_all_agents.py	0120_add_talker.py
0027_add_sqler.py	0121_add_chat_agent_talker_tool.py
0028_repopulate_all_agents.py	0122_add_talker_demo_prompt.py
0029_add_prompter.py	0123_add_whisperer.py
0030_repopulate_all_agents.py	0124_add_chat_agent_whisperer_tool.py
0031_add_flowcreator.py	0125_add_whisperer_demo_prompt.py
0032_repopulate_all_agents.py	0126_add_blenderer.py
0033_add_gitter.py	0127_add_chat_agent_blenderer_tool.py
0034_add_dockerer.py	0128_add_blenderer_demo_prompts.py
0035_add_pser.py	0129_add_editor.py
0036_add_kubernetes.py	0130_add_chat_agent_editor_tool.py
0037_add_apirer.py	0131_add_editor_demo_prompts.py
0038_add_jenkinser.py	0132_add_grepper.py
0039_add_crawler.py	0133_add_chat_agent_grepper_tool.py
0040_add_summarizer.py	0134_add_grepper_demo_prompts.py
0041_add_flowhypervisor.py	0135_add_globber.py
0042_add_mouser.py	0136_add_chat_agent_globber_tool.py
0043_agentmessage_conversation_user.py	0137_add_globber_demo_prompts.py
0044_add_counter.py	0138_add_esphomer.py
0045_add_file_interpreter.py	0139_add_chat_agent_esphomer_tool.py
0046_add_image_interpreter.py	0140_add_esphomer_demo_prompts.py
0047_add_gatewayer.py	0141_add_mcp_doctor.py
0048_add_gateway_relayer.py	0142_add_chat_agent_mcp_doctor_tool.py
0049_add_node_manager.py	0143_add_mcp_doctor_demo_prompt.py
0050_add_file_creator.py	0144_insert_java17_maven_demo_prompt_at_5.py
0051_add_file_extractor.py	0145_insert_createsuperuser_wizard_prompt_at_1.py
0052_add_kyber_keygen.py	0146_update_createsuperuser_wizard_continuation.py
0053_add_kyber_cipher.py	0147_add_unreal_game_demo_prompts.py
0054_add_kyber_decipher.py	0148_add_discoverer_demo_prompts.py
0055_fix_kyber_cipher_names.py	0149_add_discoverer.py
0056_add_parametrizer.py	0150_add_chat_agent_discoverer_tool.py
0057_add_flowbacker.py	0151_retire_old_messaging_add_telegrammer.py
0058_add_barrier.py	0152_add_instant_messaging_doctor.py
0059_add_j_decompiler.py	0153_add_chat_agent_instant_messaging_doctor_tool.py
0060_add_agent_parametrizer_tool.py	» py
0061_add_agent_starter_tool.py	0154_add_instant_messaging_doctor_demo_prompt.py
0062_sync_agent_control_tools_and_prompts.py	0155_add_contact_scaffold_demo_prompt.py
0063_add_agent_parametrizer_prompt.py	0156_add_telegrammer_identity_demo_prompts.py
0064_add_chat_agent_run_model.py	0157_add_whatsapp_identity_demo_prompt.py
0065_add_chat_wrapped_agent_tools.py	0158_redesign_messaging_demo_prompts.py
0066_add_keyboard.py	0159_add_zavuerer.py
0067_add_multi_turn_demo_prompts.py	0160_add_chat_agent_zavuerer_tool.py
0068_add_googler_tool.py	0161_add_zavuerer_demo_prompts.py
0069_add_googler_agent.py	0162_add_zavuerer_catalog_prompts.py
0070_add_tetlatlatini.py	0163_add_zavuerer_get_key_wizard_prompt.py
0071_acpx_skills.py	0164_dedup_zavuerer_setup_wizards.py
0072_add_acpx_demo_prompts.py	0165_add_image_interpreter_triple_model_demo_prompt.py
0073_acpx_demo_gemini_uplift.py	» t.py
0074_simplify_demo_prompts.py	0166_add_video_analyzer.py
0075_add_acpx_tool_surface.py	0167_add_chat_agent_video_analyzer_tool.py
0076_add_acpxer.py	0168_add_video_analyzer_demo_prompt.py
0077_add_whatstlatlatini.py	0169_add_discoverer_cvemap_latest_demo_prompt.py
0078_add_chat_agent_keyboard.py	0170_add_nmapper.py
0079_add_chat_agent_mouser_tool.py	0171_add_chat_agent_nmapper_tool.py
0080_add_chat_agent_sleeper_tool.py	0172_add_nmapper_demo_prompts.py
0081_add_window_present_and_run_wait_tools.py	0173_add_unreal_scaffold_demo_prompt.py
0082_add_chat_agent_j_decompiler_tool.py	0174_unreal_scaffold_build_project_tip.py
0083_add_de_compressor.py	0175_prompt_category_and_dedup.py
0084_add_chat_agent_de_compressor_tool.py	0176_delete_duplicate_acpx_prompts.py
0085_add_unrealer.py	0177_add_stm32er_platformio_demo_prompt.py
0086_add_chat_agent_unrealer_tool.py	0178_add_stm32_stepwise_blink_camera_prompts.py
0087_add_unrealer_demo_prompt.py	0179_regroup_resort_prompts_no_gaps.py
0088_add_reviewer.py	0180_add_kalier_setup_wizard_prompt.py
0089_add_analyzer.py	0181_prompt_sort_rank.py
0090_add_reviewer_analyzer_demo_prompts.py	0182_add_step_by_step_section_openers.py
0091_add_playwrighter.py	0183_standardize_security_recon_prompts.py
0092_add_chat_agent_playwrighter_tool.py	0184_standardize_prompt_params_batch2.py
0093_add_windower.py	0185_standardize_prompt_params_batch3.py
0094_add_chat_agent_windower_tool.py	0186_add_chat_agent_flowcreator.py

```

├── 0187_add_flowcreator_end_to_end_wizard_prompt.py
├── 0188_add_pdfer.py
├── 0189_add_chat_agent_pdfer_tool.py
├── 0190_add_pdfer_demo_prompts.py
├── 0191_add_latexer.py
├── 0192_add_chat_agent_latexer_tool.py
├── 0193_add_latexer_demo_prompts.py
├── 0194_add_deep_research_demo_prompt.py
├── 0195_add_netspeed_calculator.py
├── 0196_add_chat_agent_netspeed_calculator_tool.py
├── 0197_add_netspeed_calculator_demo_prompt.py
├── 0198_add_pdfer_nuance_demo_prompt.py
├── 0199_dedupe_llm_program_snippet_names.py
├── 0200_whisperer_demo_prompt_sound_gate.py
├── 0201_add_voice_commands_section.py
├── 0202_add_pptxer.py
├── 0203_add_chat_agent_pptxer_tool.py
├── 0204_add_pptxer_demo_prompts.py
├── 0205_silence_window_default_3_5_seconds.py
├── 0206_qwen35_cloud_model_tag_rename.py
├── 0207_video_analyzer_content_prompts.py
├── 0208_video_demo_config_models.py
├── 0209_retire_qwen35_vision_model.py
├── 0210_strict_visual_analysis_prompt_contract.py
├── 0211_compact_state.py
├── 0212_compact_state_self_modify.py
├── __init__.py
├── monitors/
│   └── monitor_sql.py
├── opus_client/
│   ├── claude_opus_client.py
│   ├── examples.py
│   └── README.md
├── rag/
│   ├── __init__.py
│   ├── binary_guard.py
│   ├── config.py
│   ├── factory.py
│   ├── interaction.py
│   ├── interface.py
│   ├── loaders.py
│   ├── prompts.py
│   ├── retrieval.py
│   ├── splitters.py
│   ├── utils.py
│   └── chains/
│       ├── base.py
│       ├── basic.py
│       ├── history_aware.py
│       └── unified.py
├── services/
│   ├── __init__.py
│   ├── agent_contracts.py
│   ├── agent_paths.py
│   ├── filesystem.py
│   ├── flow_compiler.py
│   ├── flow_knowledge.py
│   ├── flow_spec.py
│   ├── prompt_flow_panel.py
│   └── response_parser.py
├── skills/
│   ├── __init__.py
│   ├── frontmatter.py
│   ├── harness.py
│   ├── io_contract.py
│   ├── redaction.py
│   ├── registry.py
│   └── validation.py
├── skills_pkg/
│   ├── meta/
│   │   ├── lint.py
│   │   └── schema.json
│   ├── acp_router/
│   │   └── SKILL.md
│   ├── adding_external_mcp/
│   │   ├── SKILL.md
│   │   └── references/
│   │       ├── external_mcp_catalog_format.md
│   │       ├── llm_reflection_research.md
│   │       ├── transport_guide.md
│   │       └── troubleshooting.md
│   ├── code_review/
│   │   └── SKILL.md
│   ├── create_new_agent/
│   │   └── SKILL.md
│   ├── create_new_mcp/
│   │   └── SKILL.md
│   ├── flow_making/
│   │   ├── SKILL.md
│   │   └── references/
│   │       └── flw_schema.md
│   ├── scripts/
│   │   ├── make_flow.py
│   │   └── result_to_flw.py
├── github/
│   └── SKILL.md
├── gmail/
│   └── SKILL.md
├── hello_world/
│   └── SKILL.md
├── jira/
│   └── SKILL.md
├── kali_pentest/
│   └── SKILL.md
├── notion/
│   └── SKILL.md
├── roblox_studio/
│   └── SKILL.md
├── security_audit/
│   └── SKILL.md
├── setup_new_acpx_key/
│   └── SKILL.md
├── skill_creator/
│   ├── SKILL.md
│   └── scripts/
│       └── quick_validate.py
├── slack/
│   └── SKILL.md
├── summarize/
│   └── SKILL.md
├── tlamadini_allowed_hosts_tighten/
│   └── SKILL.md
├── tlamadini_csrf_exempt_audit/
│   └── SKILL.md
├── tlamadini_exec_report_row_adder/
│   └── SKILL.md
├── tlamadini_flow_from_objective/
│   └── SKILL.md
├── tlamadini_flw_doctor/
│   └── SKILL.md
├── tlamadini_new_acp_agent/
│   └── SKILL.md
├── tlamadini_planner_trace_replay/
│   └── SKILL.md
├── tlamadini_static_version_bumper/
│   └── SKILL.md
├── todoist/
│   └── SKILL.md
├── trello/
│   └── SKILL.md
├── weather/
│   └── SKILL.md
├── static/
│   └── agent/
│       ├── css/
│       │   ├── access_keys_wizard.css
│       │   ├── agent_page.css
│       │   ├── agentic_control_panel.css
│       │   ├── avatar.css
│       │   ├── chat_dictation.css
│       │   ├── contacts_dialog.css
│       │   ├── context_gauge.css
│       │   ├── dialog_theme.css
│       │   ├── external_mcps_dialog.css
│       │   ├── flow_canvas.css
│       │   ├── login.css
│       │   ├── model_brain_tuning.css
│       │   ├── model_capacity.css
│       │   ├── pdf_canvas.css
│       │   ├── pdf_context_progress.css
│       │   ├── prompt_flow_panel.css
│       │   ├── skills_dialog.css
│       │   ├── tools_dialog.css
│       │   └── welcome.css
│       ├── img/
│       │   ├── spinner.svg
│       │   ├── Tlamadini.ico
│       │   └── avatar/
│       │       ├── ec_mc.jpg
│       │       ├── ec_mo.jpg
│       │       ├── eo_mc.jpg
│       │       └── eo_mo.jpg
│       └── js/
│           ├── access_keys_wizard.js
│           ├── acp-agent-connectors.js
│           ├── acp-canvas-core.js
│           ├── acp-canvas-undo.js
│           ├── acp-connection-status.js
│           ├── acp-control-buttons.js
│           ├── acp-editor-tools.js
│           ├── acp-file-io.js
│           ├── acp-flow-snapshot.js
│           ├── acp-globals.js
│           ├── acp-layout.js
│           ├── acp-parametrizer-dialog.js
│           ├── acp-running-state.js
│           ├── acp-session.js
│           └── acp-undo-manager.js

```

```

acp-validate.js
agent_page_canvas.js
agent_page_chat.js
agent_page_context.js
agent_page_dialogs.js
agent_page_init.js
agent_page_layout.js
agent_page_pdf.js
agent_page_state.js
agent_page_ui.js
agent_control_panel.js
avatar.js
avatar_presence.js
avatar_size.js
canvas_item_dialog.js
chat_dictation.js
chat_image_paste.js
chat_page_runtime_poller.js
chat_table_contrast.js
checkbox_bulk_toggle.js
compact_costs.js
contacts_dialog.js
context_gauge.js
contextual_menus.js
dialog_policy.js
external_mcps_dialog.js
flow-canvas-interactions.js
mic_settings.js
model_brain_tuning.js
model_capacity.js
pdf_canvas_viewer.js
pdf_context_progress.js
prompt-flow-panel-model.js
prompt-flow-panel.js
release_notes_renderer.js
self_modify_switch.js
shared-runtime-dialogs.js
skills_dialog.js
tools_dialog.js
welcome_enter_default.js

pdf/
└─ canvas.html

sounds/
└─ hypervisor_alert.wav
   └─ notification.wav

vendor/
└─ frontend/
   └─ manifest.json
      └─ bootstrap/
         └─ LICENSE
            └─ dist/
               └─ css/
                  └─ bootstrap.min.css
                     └─ bootstrap.min.css.map
                  └─ js/
                     └─ bootstrap.bundle.min.js
                        └─ bootstrap.bundle.min.js.map
                           » p

      └─ highlightjs/
         └─ highlight.min.js
            └─ LICENSE

      └─ languages/
         └─ bash.min.js
            └─ c.min.js
            └─ cpp.min.js
            └─ css.min.js
            └─ ini.min.js
            └─ java.min.js
            └─ javascript.min.js
            └─ kotlin.min.js
            └─ python.min.js
            └─ rust.min.js
            └─ sql.min.js
            └─ typescript.min.js
            └─ xml.min.js
            └─ yaml.min.js

      └─ styles/
         └─ atom-one-dark.min.css

      └─ jquery/
         └─ LICENSE.txt
            └─ dist/
               └─ jquery.min.js
                  └─ jquery.min.map

      └─ jquery-ui/
         └─ jquery-ui.min.js
            └─ LICENSE.txt

      └─ nunito/
         └─ 400.css
         └─ 700.css
         └─ LICENSE
            └─ files/
               └─ nunito-cyrillic-400-normal.wof
                  └─ f
                  └─ nunito-cyrillic-400-normal.wof

```

```

      └─ f2
      └─ nunito-cyrillic-700-normal.wof
         └─ f
         └─ nunito-cyrillic-700-normal.wof
            └─ f2
            └─ nunito-cyrillic-ext-400-normal
               └─ .woff
               └─ nunito-cyrillic-ext-400-normal
                  └─ .woff2
                  └─ nunito-cyrillic-ext-700-normal
                     └─ .woff
                     └─ nunito-cyrillic-ext-700-normal
                        └─ .woff2
                        └─ nunito-latin-400-normal.woff
                        └─ nunito-latin-400-normal.woff2
                        └─ nunito-latin-700-normal.woff
                        └─ nunito-latin-700-normal.woff2
                        └─ nunito-latin-ext-400-normal.wo
                           └─ ff
                           └─ nunito-latin-ext-400-normal.wo
                              └─ ff2
                              └─ nunito-latin-ext-700-normal.wo
                                 └─ ff
                                 └─ nunito-latin-ext-700-normal.wo
                                    └─ ff2
                                    └─ nunito-vietnamese-400-normal.w
                                       └─ off
                                       └─ nunito-vietnamese-400-normal.w
                                          └─ off2
                                          └─ nunito-vietnamese-700-normal.w
                                             └─ off
                                             └─ nunito-vietnamese-700-normal.w
                                                └─ off2

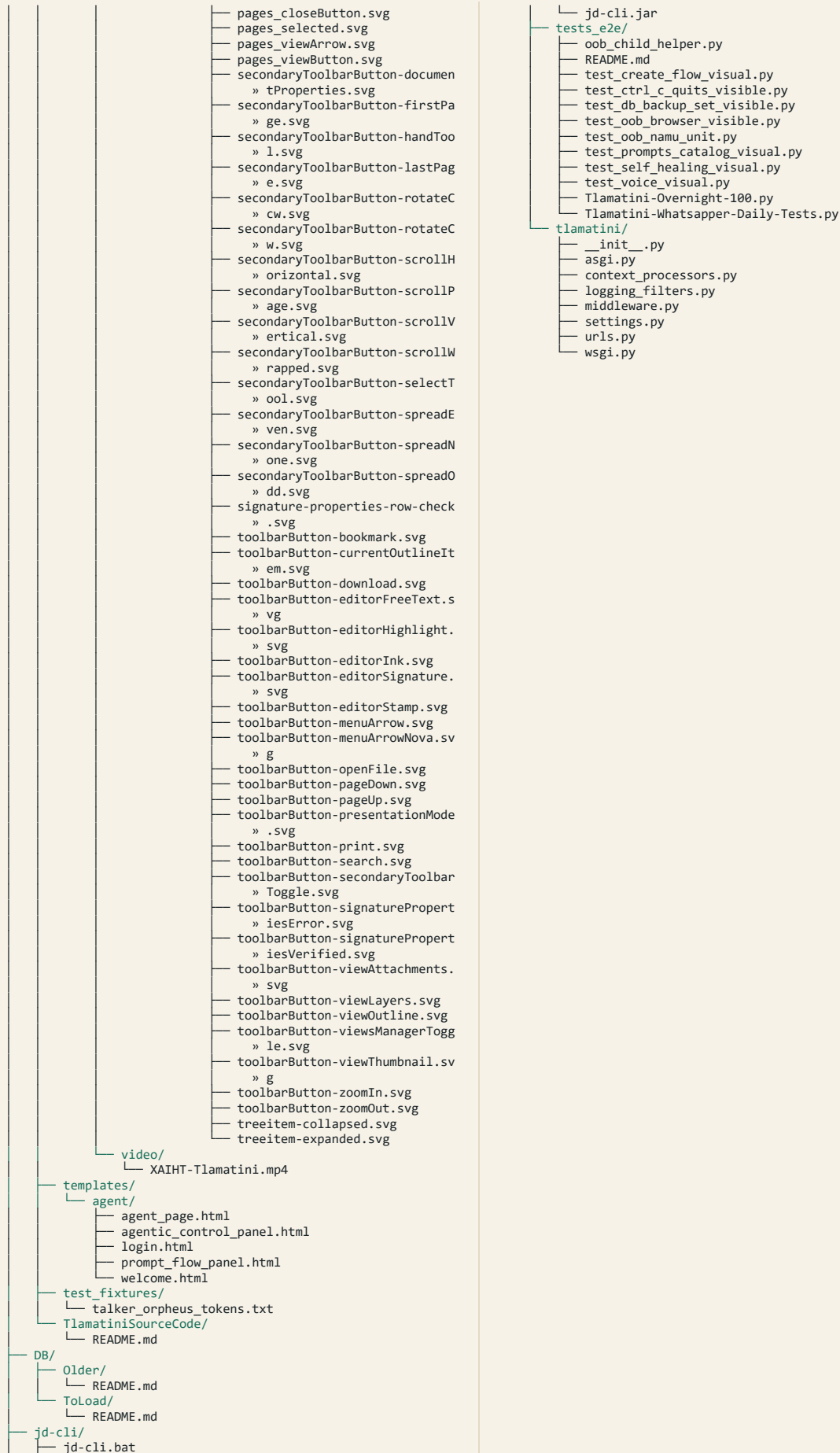
pdfjs/
└─ LICENSE
└─ VERSION
└─ build/
   └─ pdf.mjs
   └─ pdf.worker.mjs

cmaps/
└─ 78-EUC-H.bcmap
└─ 78-EUC-V.bcmap
└─ 78-H.bcmap
└─ 78-RKSJ-H.bcmap
└─ 78-RKSJ-V.bcmap
└─ 78-V.bcmap
└─ 78ms-RKSJ-H.bcmap
└─ 78ms-RKSJ-V.bcmap
└─ 83pv-RKSJ-H.bcmap
└─ 90ms-RKSJ-H.bcmap
└─ 90ms-RKSJ-V.bcmap
└─ 90msp-RKSJ-H.bcmap
└─ 90msp-RKSJ-V.bcmap
└─ 90pv-RKSJ-H.bcmap
└─ 90pv-RKSJ-V.bcmap
└─ Add-H.bcmap
└─ Add-RKSJ-H.bcmap
└─ Add-RKSJ-V.bcmap
└─ Add-V.bcmap
└─ Adobe-CNS1-0.bcmap
└─ Adobe-CNS1-1.bcmap
└─ Adobe-CNS1-2.bcmap
└─ Adobe-CNS1-3.bcmap
└─ Adobe-CNS1-4.bcmap
└─ Adobe-CNS1-5.bcmap
└─ Adobe-CNS1-6.bcmap
└─ Adobe-CNS1-UCS2.bcmap
└─ Adobe-GB1-0.bcmap
└─ Adobe-GB1-1.bcmap
└─ Adobe-GB1-2.bcmap
└─ Adobe-GB1-3.bcmap
└─ Adobe-GB1-4.bcmap
└─ Adobe-GB1-5.bcmap
└─ Adobe-GB1-UCS2.bcmap
└─ Adobe-Japan1-0.bcmap
└─ Adobe-Japan1-1.bcmap
└─ Adobe-Japan1-2.bcmap
└─ Adobe-Japan1-3.bcmap
└─ Adobe-Japan1-4.bcmap
└─ Adobe-Japan1-5.bcmap
└─ Adobe-Japan1-6.bcmap
└─ Adobe-Japan1-UCS2.bcmap
└─ Adobe-Korea1-0.bcmap
└─ Adobe-Korea1-1.bcmap
└─ Adobe-Korea1-2.bcmap
└─ Adobe-Korea1-UCS2.bcmap
└─ B5-H.bcmap
└─ B5-V.bcmap
└─ B5pc-H.bcmap
└─ B5pc-V.bcmap
└─ CNS-EUC-H.bcmap
└─ CNS-EUC-V.bcmap
└─ CNS1-H.bcmap
└─ CNS1-V.bcmap

```


- CNS2-H.bcmap
- CNS2-V.bcmap
- ETen-B5-H.bcmap
- ETen-B5-V.bcmap
- ETenms-B5-H.bcmap
- ETenms-B5-V.bcmap
- ETHK-B5-H.bcmap
- ETHK-B5-V.bcmap
- EUC-H.bcmap
- EUC-V.bcmap
- Ext-H.bcmap
- Ext-RKSJ-H.bcmap
- Ext-RKSJ-V.bcmap
- Ext-V.bcmap
- GB-EUC-H.bcmap
- GB-EUC-V.bcmap
- GB-H.bcmap
- GB-V.bcmap
- GBK-EUC-H.bcmap
- GBK-EUC-V.bcmap
- GBK2K-H.bcmap
- GBK2K-V.bcmap
- GBKp-EUC-H.bcmap
- GBKp-EUC-V.bcmap
- GBpc-EUC-H.bcmap
- GBpc-EUC-V.bcmap
- GBT-EUC-H.bcmap
- GBT-EUC-V.bcmap
- GBT-H.bcmap
- GBT-V.bcmap
- GBTpc-EUC-H.bcmap
- GBTpc-EUC-V.bcmap
- H.bcmap
- Hankaku.bcmap
- Hiragana.bcmap
- HKd1a-B5-H.bcmap
- HKd1a-B5-V.bcmap
- HKd1b-B5-H.bcmap
- HKd1b-B5-V.bcmap
- HKgccs-B5-H.bcmap
- HKgccs-B5-V.bcmap
- HKm314-B5-H.bcmap
- HKm314-B5-V.bcmap
- HKm471-B5-H.bcmap
- HKm471-B5-V.bcmap
- HKscs-B5-H.bcmap
- HKscs-B5-V.bcmap
- Katakana.bcmap
- KSC-EUC-H.bcmap
- KSC-EUC-V.bcmap
- KSC-H.bcmap
- KSC-Johab-H.bcmap
- KSC-Johab-V.bcmap
- KSC-V.bcmap
- KSCms-UHC-H.bcmap
- KSCms-UHC-HW-H.bcmap
- KSCms-UHC-HW-V.bcmap
- KSCms-UHC-V.bcmap
- KSCpc-EUC-H.bcmap
- KSCpc-EUC-V.bcmap
- LICENSE
- NWP-H.bcmap
- NWP-V.bcmap
- RKSJ-H.bcmap
- RKSJ-V.bcmap
- Roman.bcmap
- UniCNS-UCS2-H.bcmap
- UniCNS-UCS2-V.bcmap
- UniCNS-UTF16-H.bcmap
- UniCNS-UTF16-V.bcmap
- UniCNS-UTF32-H.bcmap
- UniCNS-UTF32-V.bcmap
- UniCNS-UTF8-H.bcmap
- UniCNS-UTF8-V.bcmap
- UniGB-UCS2-H.bcmap
- UniGB-UCS2-V.bcmap
- UniGB-UTF16-H.bcmap
- UniGB-UTF16-V.bcmap
- UniGB-UTF32-H.bcmap
- UniGB-UTF32-V.bcmap
- UniGB-UTF8-H.bcmap
- UniGB-UTF8-V.bcmap
- UniJIS-UCS2-H.bcmap
- UniJIS-UCS2-HW-H.bcmap
- UniJIS-UCS2-HW-V.bcmap
- UniJIS-UCS2-V.bcmap
- UniJIS-UTF16-H.bcmap
- UniJIS-UTF16-V.bcmap
- UniJIS-UTF32-H.bcmap
- UniJIS-UTF32-V.bcmap
- UniJIS-UTF8-H.bcmap
- UniJIS-UTF8-V.bcmap
- UniJIS2004-UTF16-H.bcmap
- UniJIS2004-UTF16-V.bcmap

- UniJIS2004-UTF32-H.bcmap
- UniJIS2004-UTF32-V.bcmap
- UniJIS2004-UTF8-H.bcmap
- UniJIS2004-UTF8-V.bcmap
- UniJISPro-UCS2-HW-V.bcmap
- UniJISPro-UCS2-V.bcmap
- UniJISPro-UTF8-V.bcmap
- UniJISX0213-UTF32-H.bcmap
- UniJISX0213-UTF32-V.bcmap
- UniJISX02132004-UTF32-H.bcmap
- UniJISX02132004-UTF32-V.bcmap
- UniKS-UCS2-H.bcmap
- UniKS-UCS2-V.bcmap
- UniKS-UTF16-H.bcmap
- UniKS-UTF16-V.bcmap
- UniKS-UTF32-H.bcmap
- UniKS-UTF32-V.bcmap
- UniKS-UTF8-H.bcmap
- UniKS-UTF8-V.bcmap
- V.bcmap
- WP-Symbol.bcmap
- iccs/
 - CGATS001Compat-v2-micro.icc
 - LICENSE
- standard_fonts/
 - FoxitDingbats.pfb
 - FoxitFixed.pfb
 - FoxitFixedBold.pfb
 - FoxitFixedBoldItalic.pfb
 - FoxitFixedItalic.pfb
 - FoxitSerif.pfb
 - FoxitSerifBold.pfb
 - FoxitSerifBoldItalic.pfb
 - FoxitSerifItalic.pfb
 - FoxitSymbol.pfb
 - LiberationSans-Bold.ttf
 - LiberationSans-BoldItalic.ttf
 - LiberationSans-Italic.ttf
 - LiberationSans-Regular.ttf
 - LICENSE_FOXIT
 - LICENSE_LIBERATION
- wasm/
 - jbig2.wasm
 - jbig2_nowasm_fallback.js
 - LICENSE_JBIG2
 - LICENSE_OPENJPEG
 - LICENSE_PDFJS_JBIG2
 - LICENSE_PDFJS_OPENJPEG
 - LICENSE_PDFJS_QCMS
 - LICENSE_QCMS
 - openjpeg.wasm
 - openjpeg_nowasm_fallback.js
 - qcms_bg.wasm
 - quickjs-eval.js
 - quickjs-eval.wasm
- web/
 - pdf_viewer.css
 - pdf_viewer.mjs
 - images/
 - altText_add.svg
 - altText_disclaimer.svg
 - altText_done.svg
 - altText_spinner.svg
 - altText_warning.svg
 - annotation-check.svg
 - annotation-comment.svg
 - annotation-help.svg
 - annotation-insert.svg
 - annotation-key.svg
 - annotation-newparagraph.svg
 - annotation-noicon.svg
 - annotation-note.svg
 - annotation-paperclip.svg
 - annotation-paragraph.svg
 - annotation-pushpin.svg
 - checkmark.svg
 - comment-actionsButton.svg
 - comment-closeButton.svg
 - comment-editButton.svg
 - comment-popup-editButton.svg
 - cursor-editorFreeHighlight.svg
 - cursor-editorFreeText.svg
 - cursor-editorInk.svg
 - cursor-editorTextHighlight.svg
 - editor-toolbar-delete.svg
 - editor-toolbar-edit.svg
 - findbarButton-next.svg
 - findbarButton-previous.svg
 - gv-toolbarButton-download.svg
 - loading-icon.gif
 - loading.svg
 - messageBar_closingButton.svg
 - messageBar_info.svg
 - messageBar_warning.svg





Tlamatini

one who knows

Conceived, designed and built by **Angela López Mendoza**. Every fact in this dossier was counted from the source tree when it was generated; regenerate it at any time with **refresh_project_docs.py** and it will tell the truth again.

Set in Palatino Linotype, Segoe UI and Consolas. Generated October 10, 2026 at 13:33 (UTC-06:00) from commit 0e6fc72d. MIT license - github.com/XAIHT/Tlamatini

